

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a0a6216face8134d5.jsonl`  
- SHA-256: `a2387a7a5c592031f4751b679c55108910cf19a8de6e1bd87e53a218fd2408a7`  
- Source modified: `2026-06-10T16:27:42+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.750Z)

You are doing a DEEP correctness review of the parsing and ingestion subsystem of muneem (C:/Users/avidu/Projects/muneem, Python, src/muneem). This subsystem turns Indian bank statement PDFs (HDFC, Axis, AU Bank savings + credit card) into ledger rows. Architecture: a bank-agnostic concept schema (parsers/concepts.py), a generic column-to-concept mapping engine (parsers/engine.py) parameterized by issuer profiles (parsers/profiles.py), coordinate clustering for semi-ruled tables (parsers/clustering.py), a shared savings path (parsers/savings.py), per-issuer parsers (hdfc.py, axis.py, au.py), and an arithmetic verification oracle (parsers/validation.py) that gates persistence: per-row running-balance walk + total reconcile + SUMMARY bookend. Ingestion providers live in src/muneem/ingestion (gmail.py full OAuth read-only; imap.py and outlook.py experimental).

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Read EVERY file under src/muneem/parsers/ and src/muneem/ingestion/ completely, plus tests/parsers/*.py and tests/synthpdf.py to understand intended behavior.

Hunt specifically for:

- Money arithmetic: float vs Decimal use anywhere amounts are parsed, summed, or compared; rounding/epsilon comparisons in the balance walk; sign/direction errors in the balance-delta reconstruction (deriving debit/credit from the running-balance sign).
- Date parsing: Indian DD/MM/YYYY vs MM/DD ambiguity, two-digit years, locale assumptions, statement-period boundary handling.
- Oracle gating: ANY code path where rows can be persisted without passing the verifier; how 'unreconciled' status is handled; whether partial persistence can occur on exception mid-insert.
- Clustering: y/x tolerance assumptions, dropped or merged lines (the known Axis May-2025 dropped-line failure ? characterize WHY), multi-line description handling, header/footer bleed.
- Engine/profile matching: false-positive parser selection (get_parser text heuristics), L0/L2 fallback correctness, column-order ambiguity.
- Dedup: sha256-based document dedup interplay with re-parsing after a parser fix (can a fixed parser re-ingest a previously failed doc?).
- Ingestion: OAuth scope breadth, token storage, pagination, attachment-size handling, MIME walking, error handling; confirm imap/outlook are genuinely marked experimental and unreachable from the CLI.
- Anything where a plausible real statement layout would silently produce wrong rows that still reconcile (the worst case).

Report ONLY real, defensible findings with exact file:line and quoted (non-sensitive) code evidence; honest severities; include genuinely-worth-fixing medium/low items. Structured output only.

2. user (2026-06-10T16:13:33.301Z)

```
src\muneem\__init__.py
src\muneem\config.py
src\muneem\offers.py
src\muneem\ocr.py
src\muneem\ingestion\__init__.py
src\muneem\ingestion\base.py
src\muneem\ingestion\gmail.py
src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\extract.py
src\muneem\parsers\savings.py
src\muneem\log.py
src\muneem\redaction.py
src\muneem\password_rules.py
src\muneem\secrets.py
src\muneem\statement_unlock.py
src\muneem\parsers\engine.py
src\muneem\unlock.py
src\muneem\db_key.py
src\muneem\parsers\base.py
src\muneem\ingestion\imap.py
src\muneem\ingestion\outlook.py
src\muneem\cli.py
src\muneem\recovery.py
src\muneem\errors.py
src\muneem\db.py
src\muneem\migrations.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py
```

3. user (2026-06-10T16:13:34.199Z)

```
tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\test_offers.py
tests\test_repo_hygiene.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_registry.py
tests\parsers\test_validation.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_local_axis.py
tests\parsers\test_au.py
tests\parsers\test_local_au.py
tests\synthpdf.py
tests\test_regression_synth.py
tests\parsers\test_savings.py
tests\test_eval_cli.py
```

```

tests\conftest.py
tests\test_cli.py
tests\test_db_crypto.py
tests\parsers\test_parse_errors.py
tests\test_errors.py
tests\test_log.py
tests\test_redaction.py
tests\test_password_rules.py
tests\test_secrets.py
tests\test_statement_unlock.py
tests\test_db_crypto_ops.py
tests\test_verifier_properties.py
tests\test_dependency_licenses.py
tests\test_precommit_hook.py
tests\test_recovery.py
tests\test_db_invariants.py
tests\test_migrations.py
tests\test_provenance.py

```

4. user (2026-06-10T16:13:39.002Z)

```

1      """The canonical *concept schema* ? one bank-agnostic vocabulary every extractor
targets.
2
3      This is the **contract** at the heart of the concept-core parser architecture (see
4      [`docs/parser-architecture.md`](../../docs/parser-architecture.md) §3): a parser's
job is to
5      *map this bank's columns/fields onto these concepts, then let the verifier check them.*
Because the
6      concepts are layout-independent, a format change becomes "re-map onto the same
concepts," not
7      "rewrite the parser."
8
9      These are pure data ? no PDF, no DB, no logic ? so they import nothing and are safe to
use anywhere.
10     The verifier (`muneem.parsers.validation`) consumes them; bank extractors produce
them.
11     """
12
13     from __future__ import annotations
14
15     from dataclasses import dataclass
16
17
18     @dataclass
19     class ConceptRow:
20         """One normalized transaction, in the bank-agnostic vocabulary.
21
22         ``debit`` is money out (withdrawal / spend / Dr); ``credit`` is money in (deposit /
Cr).
23         Exactly one is typically non-``None`` on a real row, but both may be ``None`` for a
24         non-amount marker row. ``balance`` is the running account balance *after* this row,
or
25         ``None`` when the statement carries no per-row balance.
26
27         **Credit cards** (no running balance) map onto the same row: put the charge in
``debit`` or
28         the payment/refund in ``credit`` according to the printed Dr/Cr direction, and leave
29         ``balance=None``. That keeps one row type for the whole ladder; total reconciliation
(not the
30         per-row walk) is what validates a balance-less statement.
31         """
32
33         txn_date: str
34         narration: str

```

```

35         debit: float | None = None
36         credit: float | None = None
37         balance: float | None = None
38         value_date: str | None = None
39         reference: str | None = None
40
41
42     @dataclass
43     class StatementConcepts:
44         """Statement-level concepts ? the figures a bank prints in its own SUMMARY / header.
45
46         Every field is optional: ``None`` means "this statement didn't expose it," which the
verifier
47         treats as *not checkable* (never as a failure). ``debit_amount`` / ``credit_amount``
are the
48         bank's own printed totals; they are recorded here for completeness but are not yet
asserted by
49         :func:`muneem.parsers.validation.verify` (the current bookend checks counts +
opening/closing,
50         matching the established HDFC behavior).
51         """
52
53         opening_balance: float | None = None
54         closing_balance: float | None = None
55         debit_count: int | None = None
56         credit_count: int | None = None
57         debit_amount: float | None = None
58         credit_amount: float | None = None
59         account_number: str | None = None
60         period_start: str | None = None
61         period_end: str | None = None
62         currency: str | None = None
63
64
65     __all__ = ["ConceptRow", "StatementConcepts"]
66

```

5. user (2026-06-10T16:13:39.056Z)

```

1     """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3     This is the *generic-first* half of the concept-core architecture (see
4     [docs/parser-architecture.md](`../..../docs/parser-architecture.md`) §4, §6).
5     Bank-agnostic code that takes a cell-matrix (the shape pdfplumber.extract_tables()
returns,
6     or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7     via a cheap-first ladder, with a profile supplying priors:
8
9     * L0 ? header lexicon. Match header-row labels against the profile's concept?label
aliases.
10    * profile-order. If there is no header, fall back to the profile's known column
order (a prior).
11    * L1/L2 ? content + search. Classify columns by content (dates, money), then
enumerate the
12    plausible amount-column ? {debit, credit, balance} assignments and keep the one the
13    reconciliation oracle endorses ? the self-identifying balance column, found by
which mapping
14    makes the math work* rather than by its label.
15
16    The engine is family-agnostic: it is parameterized by a :class:`FamilySpec` (the
target concept
17    names, which are date/amount/balance, and a typed-row builder) and an injected oracle`
18    (`rows -> bool`). Only the bank family ships today; an instrument family
(equity/MF/ETF/bond ? see

```

```

19     the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20     value; provenance is the winning layer + column map.
21     """
22
23     from __future__ import annotations
24
25     import itertools
26     import re
27     from collections.abc import Callable, Sequence
28     from dataclasses import dataclass, field
29
30     from .concepts import ConceptRow
31     from .validation import parse_amount
32
33     Cell = object # a table cell: str | None (we coerce defensively)
34     Row = Sequence[Cell]
35
36     # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
37     _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
38         re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy
39         re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy
40     )
41     # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
42     # for an amount column during content classification.
43     _MONEY_RE = re.compile(r"(?i)^\d{1,2}(?:\s*(?:cr|dr))?$")
44     _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
45
46     # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
47     # lower-cased header labels matched exactly (not by substring) during L0.
48     _BANK_ALIASES: dict[str, tuple[str, ...]] = {
49         "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
50         "value_date": ("value date", "val date", "value dt"),
51         "narration": (
52             "narration",
53             "particulars",
54             "description",
55             "transaction details",
56             "details",
57             "remarks",
58             "transaction",
59             "transaction remarks",
60         ),
61         "reference": (
62             "ref",
63             "reference",
64             "ref no",
65             "reference no",
66             "cheque no",
67             "chq no",
68             "instrument no",
69         ),
70         "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
71         "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
72         "balance": (
73             "balance",
74             "closing balance",
75             "running balance",
76             "closing bal",
77             "available balance",
78         ),
79     }

```

```

80
81
82     def _text(cell: Cell) -> str:
83         return "" if cell is None else str(cell).strip()
84
85
86     def _norm(label: str) -> str:
87         return re.sub(r"\s+", " ", label.strip().lower())
88
89
90     def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
91         return any(p.match(text) for p in patterns)
92
93
94     def _looks_like_money(text: str) -> bool:
95         return bool(_MONEY_RE.match(text))
96
97
98     @dataclass
99     class FamilySpec:
100         """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
101
102         The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
103         future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
104         ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105         """
106
107         name: str
108         concepts: tuple[str, ...]
109         date_concepts: tuple[str, ...] # in order: the first is the primary (row-gating)
date
110         amount_concepts: tuple[str, ...] # money columns; the non-balance ones are the
"flows"
111         balance_concept: str | None # the running-balance concept, if the family has one
112         text_concepts: tuple[str, ...] # free-text columns; the first gets the widest text
column
113         default_aliases: dict[str, tuple[str, ...]]
114         build: Callable[[list[dict[str, str]]], list]
115
116
117     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
118         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134
135     BANK_FAMILY = FamilySpec(
136         name="bank",

```

```

137     concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
138     date_concepts=("txn_date", "value_date"),
139     amount_concepts=("debit", "credit", "balance"),
140     balance_concept="balance",
141     text_concepts=("narration", "reference"),
142     default_aliases=_BANK_ALIASES,
143     build=_build_bank_rows,
144 )
145
146
147 @dataclass
148 class MappingResult:
149     """The outcome of mapping one table: typed rows + how we got there (provenance)."""
150
151     rows: list # typed rows from family.build (e.g. list[ConceptRow])
152     mapped: list[
153         dict[str, str]
154     ] # per-row {concept: raw cell}, incl. any profile-declared extra columns
155     column_map: dict[str, int] # the winning concept -> column-index assignment
156     layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
157     verified: bool # did the oracle endorse it? (False if no oracle was supplied)
158
159
160 @dataclass
161 class Profile:
162     """Declarative per-issuer knowledge ? *data, not code* (architecture §6).
163
164     Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
165     accepted date shape, a positional fallback when there's no header, table anchors,
account-number
166     patterns, quirks, and (placeholder, wiring parked) the password rule.
167     """
168
169     name: str
170     aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
171     date_patterns: tuple[re.Pattern, ...] = ()
172     column_order: tuple[str, ...] | None = None
173     table_anchors: tuple[str, ...] = ()
174     account_patterns: tuple[re.Pattern, ...] = ()
175     quirks: frozenset[str] = frozenset()
176     password_rule: str | None = None # data only; sourcing is parked (DESIGN §8.2)
177
178     def first_account(self, text: str) -> str | None:
179         for rx in self.account_patterns:
180             m = rx.search(text)
181             if m:
182                 return m.group(1).strip()
183         return None
184
185
186     def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
187         """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
188         index: dict[str, str] = {}
189         for concept, labels in family.default_aliases.items():
190             for label in labels:
191                 index[_norm(label)] = concept
192         for concept, labels in profile.aliases.items():
193             for label in labels:
194                 index[_norm(label)] = concept
195         return index
196
197

```

```

198 def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
199     return profile.date_patterns or _GENERIC_DATE_PATTERNS
200
201
202 def _detect_header(
203     rows: list[list[Cell]], alias_index: dict[str, str]
204 ) -> tuple[int | None, dict[str, int]]:
205     """Find the header row (most exact alias hits, >=2) and its concept->column map."""
206     best_idx: int | None = None
207     best_map: dict[str, int] = {}
208     for i, row in enumerate(rows):
209         cmap: dict[str, int] = {}
210         for col, cell in enumerate(row):
211             concept = alias_index.get(_norm(_text(cell)))
212             if concept and concept not in cmap:
213                 cmap[concept] = col
214             if len(cmap) > len(best_map):
215                 best_map, best_idx = cmap, i
216     if len(best_map) < 2:
217         return None, {}
218     return best_idx, best_map
219
220
221 def _is_balance_marker(row: Row) -> bool:
222     return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)
223
224
225 def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
226     if date_col is not None:
227         cell = row[date_col] if date_col < len(row) else None
228         return _matches_date(_text(cell), patterns)
229     return any(_matches_date(_text(c), patterns) for c in row)
230
231
232 def _classify_columns(
233     data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
234 ) -> dict[int, str]:
235     """Tag each column date / amount / text / empty by majority content."""
236     kinds: dict[int, str] = {}
237     for c in range(ncols):
238         cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
239         if not cells:
240             kinds[c] = "empty"
241             continue
242         thresh = max(1, int(len(cells) * 0.6))
243         if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
244             kinds[c] = "date"
245         elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
246             kinds[c] = "amount"
247         else:
248             kinds[c] = "text"
249     return kinds
250
251
252 def _base_map(
253     col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
254 ) -> dict[str, int]:
255     """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.
256
257     Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
258     goes to the family's first text concept (if any). Amount columns are left to the
search.

```



```

259     """
260     base: dict[str, int] = {}
261     date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
262     text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
263     for concept, col in zip(family.date_concepts, date_cols, strict=False):
264         base[concept] = col
265     if text_cols and family.text_concepts:
266         base[family.text_concepts[0]] = max(
267             text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
268         )
269     return base
270
271
272     def _search_candidates(
273         base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
274     ) -> list[tuple[str, dict[str, int]]]:
275         """Enumerate plausible amount-column ? concept assignments (the L2 search space).
276
277         Family-agnostic: the "flows" are the family's amount concepts minus its balance
278         concept (for
279         banks: debit, credit). We try each amount column as the balance and assign the rest
280         to the
281         flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
282         Oct).
283         """
284         amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
285         flows = [c for c in family.amount_concepts if c != family.balance_concept]
286         bal = family.balance_concept
287         cands: list[dict[str, int]] = []
288         if bal and len(amount_cols) >= len(flows) + 1:
289             for bcol in amount_cols:
290                 rest = [c for c in amount_cols if c != bcol]
291                 for assign in itertools.permutations(rest, len(flows)):
292                     cands.append(**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
293             if flows and len(amount_cols) >= len(flows):
294                 for assign in itertools.permutations(amount_cols, len(flows)):
295                     cands.append(**base, **dict(zip(flows, assign, strict=True)))
296         return [("L2", c) for c in cands[:24]]
297
298
299     def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
300         out: list[dict[str, str]] = []
301         for row in data:
302             out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
303             col_map.items()})
304         return out
305
306
307     def map_table(
308         table: Sequence[Row] | None, # Sequence (covariant) so list[list[str | None]]
309         callers fit
310         profile: Profile,
311         family: FamilySpec = BANK_FAMILY,
312         *,
313         oracle: Callable[[list], bool] | None = None,
314     ) -> MappingResult | None:
315         """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
316         candidates.
317
318         Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
319         With an
320         ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
321         the first
322         structurally-complete candidate is returned unverified (the caller reconciles

```

```

separately).
315         Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
316         was supplied but nothing reconciled.
317         """
318         if not table:
319             return None
320         rows_in = [list(r) for r in table if r is not None]
321         if not rows_in:
322             return None
323         alias_index = _alias_index(profile, family)
324         patterns = _date_patterns(profile)
325
326         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
327         start = header_idx + 1 if header_idx is not None else 0
328         date_col_hint = col_map_hdr.get("txn_date")
329         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
330             date_col_hint = profile.column_order.index("txn_date")
331
332         data = [
333             r
334             for r in rows_in[start:]
335             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
336         ]
337         if not data:
338             return None
339         ncols = max(len(r) for r in data)
340         col_kinds = _classify_columns(data, ncols, patterns)
341         base = _base_map(col_kinds, data, family)
342         flows = [c for c in family.amount_concepts if c != family.balance_concept]
343
344         candidates: list[tuple[str, dict[str, int]]] = []
345         if flows and all(c in col_map_hdr for c in flows): # L0: header pinned the flow
concepts
346             candidates.append(("L0", {**base, **col_map_hdr}))
347         elif col_map_hdr: # header present but didn't pin the flows: keep its hints, search
the rest
348             candidates.append(("L0", {**base, **col_map_hdr}))
349         if profile.column_order: # positional prior (no/partial header)
350             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351         candidates += _search_candidates(base, col_kinds, family)
352
353         primary_date = family.date_concepts[0] if family.date_concepts else None
354         best: MappingResult | None = None
355         for layer, cmap in candidates:
356             if primary_date and primary_date not in cmap:
357                 continue
358             if not (set(family.amount_concepts) & cmap.keys()):
359                 continue
360             mapped = _apply_map(data, cmap)
361             typed = family.build(mapped)
362             if oracle is None:
363                 return MappingResult(typed, mapped, cmap, layer, verified=False)
364             if oracle(typed):
365                 return MappingResult(typed, mapped, cmap, layer, verified=True)
366             if best is None:
367                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
368         return best
369
370
371 __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372

```

6. user (2026-06-10T16:13:44.366Z)

```
1      """The verifier ? reconciliation as a layout-independent truth oracle.
2
3      Reconciliation proves a parse is complete and correct without inspecting any sensitive
value**:
4      if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5      parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6      money ? extraction proposes, reconciliation disposes (see
7      [docs/parser-architecture.md](../docs/parser-architecture.md) $2, $5).
8
9      This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * per-row walk ? reconcile_running_balance: balance[i] == balance[i-1] ? debit
+ credit.
12      * total reconcile ? reconcile_total: opening + ?credit ? ?debit == closing
(work even
13      with no per-row balance, e.g. Axis savings).
14      * bookend ? verify: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * selection ? select_reconciling: among candidate concept mappings, pick the one
arithmetic
16      endorses (the seam the L2 mapping search plugs into).
17      * confidence ? Confidence + BookendReport.confidence: a recorded verdict,
not an
18      assumption.
19
20      Reports carry counts / booleans / a verdict only ? never an amount.
21      """
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
27      from dataclasses import dataclass
28      from enum import StrEnum
29
30      from .concepts import ConceptRow, StatementConcepts
31
32      _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35      def parse_amount(value: object) -> float | None:
36          """Parse a statement money cell to a float.
37
38          Returns None only for a genuinely absent cell (None / empty / "" /
non-numeric); "0.00" parses to 0.0. This deliberately distinguishes
39          absent from zero ? unlike base.safe_amount, which maps "0.00" to
40          None and would corrupt a real zero balance and the reconciliation math.
41          """
42
43          if value is None:
44              return None
45          text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46          if text in ("", "-"):
47              return None
48          try:
49              return float(text)
50          except ValueError:
51              return None
52
53
54      @dataclass
```

```

55 class ReconcileReport:
56     """Result of a running-balance walk. Counts only, never an amount."""
57
58     n: int # rows considered
59     compared: int # consecutive (balance-bearing) pairs actually checked
60     breaks: int
61     first_break: int | None = None
62
63     @property
64     def ok(self) -> bool:
65         return self.compared > 0 and self.breaks == 0
66
67
68     def reconcile_running_balance(
69         entries: Sequence[tuple[float | None, float | None, float | None]],
70         tol: float = 0.01,
71     ) -> ReconcileReport:
72         """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
73
74         ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
75         ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
76         is skipped (can't anchor on it). Because the walk spans the whole statement, a
77         clean result also rules out any row missed *between* two balances ? which is the
78         failure mode silent ``extract_tables()`` row-drops would otherwise hide.
79         """
80         breaks = 0
81         first_break: int | None = None
82         compared = 0
83         prev: float | None = None
84         for i, (debit, credit, balance) in enumerate(entries):
85             if balance is None:
86                 continue
87             if prev is not None:
88                 expected = prev - (debit or 0.0) + (credit or 0.0)
89                 compared += 1
90                 if abs(expected - balance) > tol:
91                     breaks += 1
92                 if first_break is None:
93                     first_break = i
94             prev = balance
95         return ReconcileReport(
96             n=len(entries), compared=compared, breaks=breaks, first_break=first_break
97         )
98
99
100     def _row_entries(
101         rows: Sequence[ConceptRow],
102     ) -> list[tuple[float | None, float | None, float | None]]:
103         """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
104         consumes."""
105         return [(r.debit, r.credit, r.balance) for r in rows]
106
107
108     def reconcile_total(
109         rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
110     ) -> bool:
111         """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
112
113         This is the oracle for statements with **no reliable per-row balance** (e.g. Axis
114         savings):
115         it checks the whole statement against its own opening/closing bookends without
116         needing each
117         row's running balance. ``opening`` and ``closing`` come from the bank's
118         SUMMARY/header
119         (``StatementConcepts``); the caller must have them.

```

```

116         """
117         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
118         return abs((opening + delta) - closing) <= tol
119
120
121     class Confidence(StrEnum):
122         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).
123
124         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
125         a check failed (or none could run); the parse is suspect and is quarantined rather
than trusted.
126         """
127
128         RECONCILED = "reconciled"
129         UNRECONCILED = "unreconciled"
130
131
132     @dataclass
133     class BookendReport:
134         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
135
136         ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
137         requires the per-row balance walk *plus* every available check to pass.
138         """
139
140         n: int
141         reconciles: bool
142         debit_count_ok: bool | None = None
143         credit_count_ok: bool | None = None
144         opening_ok: bool | None = None
145         closing_ok: bool | None = None
146
147         @property
148         def ok(self) -> bool:
149             checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
150             return self.n > 0 and self.reconciles and all(c is True for c in checks)
151
152         @property
153         def confidence(self) -> Confidence:
154             return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
155
156
157     def verify(
158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.
161
162         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
163         extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
164         figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
165         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
166         """
167         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
168         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
169         if concepts.debit_count is not None:
170             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==

```

```

concepts.debit_count
171         if concepts.credit_count is not None:
172             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
173         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
174             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
175         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
176             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
177             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
178         return BookendReport(
179             n=len(rows),
180             reconciles=reconciles,
181             debit_count_ok=debit_count_ok,
182             credit_count_ok=credit_count_ok,
183             opening_ok=opening_ok,
184             closing_ok=closing_ok,
185         )
186
187
188     def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
189         """Among candidate concept-row mappings, return the index of the first that
reconciles.
190
191         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215

```

7. user (2026-06-10T16:13:44.909Z)

```

1     """Coordinate clustering ? turn positioned words into a clean cell-matrix.
2
3     When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned
statement (the
4     Axis case: a date merged into the narration cell, column counts drifting between
vintages), we read
5     **word positions** directly. Words are grouped into visual rows by their ``top``; column

```

```

boundaries
6     are inferred from the vertical whitespace **shared across the transaction rows**; and
each word is
7     dropped into the column its x-centre falls in. Because columns come from geometry, a
leading date
8     sits in its *own* column instead of being glued to the narration.
9
10    Two robustness lessons baked in:
11
12    * **Infer columns from transaction-like rows only.** Headers, address blocks, and
summaries span the
13    page and would erase the inter-column whitespace. A caller-supplied ``keep_line``
predicate (e.g.
14    "has a date *and* a money token") restricts both column inference and the emitted
matrix to the
15    transaction band.
16    * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
empty
17    in *most* rows; a single long narration can't collapse two columns.
18
19    Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's
oracle still
20    decides whether the result is trustworthy. Pure geometry in, strings out ? no value is
logged.
21    ""
22
23    from __future__ import annotations
24
25    from collections.abc import Callable, Sequence
26
27    Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
28
29
30    def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
31        """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
32        lines: list[list[Word]] = []
33        anchor_top: float | None = None
34        for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
35            if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
36                lines[-1].append(w)
37            else:
38                lines.append([w])
39                anchor_top = w["top"]
40        return [sorted(line, key=lambda w: w["x0"]) for line in lines]
41
42
43    def infer_columns(
44        lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
45    ) -> list[tuple[float, float]]:
46        """Infer column x-extents from a coverage histogram over ``lines``.
47
48        An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
columns are
49        the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
50        to a few full-width lines: it thresholds on the *fraction* of rows, not on any
single one.
51        ""
52        words = [w for line in lines for w in line]
53        if not words:
54            return []
55        x0 = min(float(w["x0"]) for w in words)
56        x1 = max(float(w["x1"]) for w in words)
57        nbins = max(1, int((x1 - x0) / bin_w) + 1)

```

```

58     cover = [0] * nbins
59     for line in lines:
60         marked: set[int] = set()
61         for w in line:
62             a = int((float(w["x0"]) - x0) / bin_w)
63             b = int((float(w["x1"]) - x0) / bin_w)
64             marked.update(range(max(0, a), min(nbins - 1, b) + 1))
65         for k in marked:
66             cover[k] += 1
67     thresh = max(1.0, min_frac * len(lines))
68     filled = [c >= thresh for c in cover]
69     cols: list[tuple[float, float]] = []
70     start: int | None = None
71     for i, f in enumerate(filled):
72         if f and start is None:
73             start = i
74         elif not f and start is not None:
75             cols.append((x0 + start * bin_w, x0 + i * bin_w))
76             start = None
77     if start is not None:
78         cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
79     return cols
80
81
82     def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
83         for i, (a, b) in enumerate(columns):
84             if a <= center <= b:
85                 return i
86         return min(
87             range(len(columns)),
88             key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
89         )
90
91
92     def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) ->
list[str]:
93         """Place each word into its column; join words that share a column (in x order)."""
94         cells = [""] * len(columns)
95         for w in line:
96             idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
97             cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
98         return cells
99
100
101     def cluster_to_matrix(
102         words_by_page: Sequence[Sequence[Word]],
103         *,
104         line_tol: float = 2.5,
105         min_frac: float = 0.3,
106         keep_line: Callable[[list[str]], bool] | None = None,
107     ) -> list[list[str]]:
108         """Cluster words across pages into one cell-matrix with consistent columns.
109
110         ``keep_line`` (texts of a line ? bool) restricts both column inference and the
output to the
111         transaction band ? without it, full-width headers/footers swamp the column
structure. Columns
112         are inferred once across all kept lines (a statement keeps its layout); rows are
emitted in
113         page-then-top order.
114         """
115         lines: list[list[Word]] = []
116         for page in words_by_page:
117             lines.extend(group_lines(page, line_tol))
118         if keep_line is not None:

```



```

119         lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
120     if not lines:
121         return []
122     columns = infer_columns(lines, min_frac=min_frac)
123     if not columns:
124         return []
125     return [line_to_cells(line, columns) for line in lines]
126
127
128     __all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]
129

```

8. user (2026-06-10T16:13:48.966Z)

```

1     """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
S6).
2
3     A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4     concept?label aliases, the accepted date shape, a positional fallback when there's no
header,
5     table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
an issuer is
6     ideally *adding a profile*, not writing a parser.
7
8     `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
coordinate-
9     clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
table (see the
10     `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
engine-driven
11     parsers (coordinate clustering ? generic mapping ? total reconcile).
12     """
13
14     from __future__ import annotations
15
16     import re
17
18     from .engine import Profile
19
20     AXIS_PROFILE = Profile(
21         name="axis",
22         aliases={
23             # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
| Balance |
24             # Other Information. "Transaction" is the narration column (not the date).
25             "narration": ("transaction", "particulars"),
26             "debit": ("withdrawals", "withdrawal", "debit"),
27             "credit": ("deposits", "deposit", "credit"),
28             "balance": ("balance",),
29         },
30         date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),), # dd-mm-yyyy or
dd/mm/yyyy
31         # Layout / column-count varies between vintages, so there is NO reliable
column_order ? the
32         # generic content search + total reconcile do the work. Broadened account pattern:
the old
33         # `Account No:` regex missed real statements.
34         account_patterns=(
35             re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?s*(\d{6,})", re.IGNORECASE),
36             re.compile(r"A/c\s*No\.\.?s*[:\s]*(\d{6,})", re.IGNORECASE),
37         ),
38         quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39     )
40
41     AUBANK_PROFILE = Profile(

```

```

42     name="aubank",
43     aliases={
44         "narration": (
45             "transaction",
46             "particulars",
47             "description",
48             "narration",
49             "transaction details",
50         ),
51         "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52         "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53         "balance": ("balance", "closing balance", "running balance"),
54     },
55     # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56     date_patterns=(
57         re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58         re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59     ),
60     account_patterns=(
61         re.compile(r"Account\s*(?:No|Number)\s*[:.\-]?s*(\d{6,})", re.IGNORECASE),
62         re.compile(r"A/c\s*No\.\s*[:.\-]?s*(\d{6,})", re.IGNORECASE),
63     ),
64     quirks=frozenset({"engine-driven"}),
65 )
66
67 HDFC_PROFILE = Profile(
68     name="hdfc",
69     aliases={
70         "debit": ("withdrawal", "withdrawals"),
71         "credit": ("deposit", "deposits"),
72         "balance": ("closing balance",),
73     },
74     date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
75     table_anchors=("Narration", "Closing Balance"),
76     account_patterns=(re.compile(r"Account\s*Number\s*[:.\-]?s*(\d{6,})",
re.IGNORECASE),),
77     # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78     # by word-position clustering rather than through the generic cell-matrix engine.
79     quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80 )
81
82 __all__ = ["AXIS_PROFILE", "AUBANK_PROFILE", "HDFC_PROFILE"]
83

```

9. user (2026-06-10T16:13:49.417Z)

```

1     """"Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3     Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4     and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6     1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7     coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8     2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9     3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10     vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12     A statement splits its ledger across several extracted tables (page breaks), so we map

```

```

each table
13     on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14     cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15     can't yet read returns ``verified=False`` and the caller persists nothing ? never
misabeled rows.
16
17     No statement content is logged; only counts/booleans travel out.
18     """
19
20     from __future__ import annotations
21
22     import re
23     from dataclasses import dataclass, field
24     from pathlib import Path
25
26     import pdfplumber
27
28     from .clustering import cluster_to_matrix
29     from .concepts import ConceptRow, StatementConcepts
30     from .engine import Profile, map_table
31     from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33     _MONEY_TOKEN = re.compile(r"^~?[\d,]+\.\d{2}(:cr|dr)?$", re.IGNORECASE)
34     _OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
35     _CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
36
37
38     @dataclass
39     class SavingsParse:
40         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42         account_number: str
43         rows: list[ConceptRow] = field(default_factory=list)
44         verified: bool = False
45
46
47     def parse_statement_summary(text: str) -> StatementConcepts:
48         """Read opening/closing balance from statement text (for the total-reconcile
oracle).
49
50         Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
51         """
52         opening = _OPENING_RE.search(text)
53         closing = _CLOSING_RE.search(text)
54         return StatementConcepts(
55             opening_balance=parse_amount(opening.group(1)) if opening else None,
56             closing_balance=parse_amount(closing.group(1)) if closing else None,
57         )
58
59
60     def txn_line_predicate(profile: Profile):
61         """A line is transaction-like if it carries one of the profile's dates *and* a money
token.
62
63         Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
64         address blocks that would otherwise wreck column inference).
65         """
66         patterns = profile.date_patterns
67
68         def keep(texts: list[str]) -> bool:

```

```

69         has_date = any(any(p.match(t) for p in patterns) for t in texts)
70         return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
71
72     return keep
73
74
75     def reconcile_oracle(opening: float | None, closing: float | None):
76         """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""
77
78         def check(rows: list[ConceptRow]) -> bool:
79             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
80                 return True
81             if rows and opening is not None and closing is not None:
82                 return reconcile_total(rows, opening, closing)
83             return False
84
85         return check
86
87
88     def _reconstruct_by_balance_delta(
89         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
90     ) -> list[ConceptRow]:
91         """Recover debit/credit from the running balance when columns won't split cleanly.
92
93         Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
94         it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
95         single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
96         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
97         a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance
98         to fix the first row's direction; returns ``[]`` if it can't form a clean series.
99         """
100         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
101         rows = [[(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
102         if not rows or opening is None:
103             return []
104         ncols = max(len(r) for r in rows)
105
106         def is_date(s: str) -> bool:
107             return any(p.match(s) for p in patterns)
108
109         def money(s: str) -> float | None:
110             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
111
112         # date column = the leftmost of the (first few) columns with the most date cells
113         date_counts: dict[int, int] = {}
114         for r in rows:
115             for c in range(min(ncols, 3)):
116                 if c < len(r) and is_date(r[c]):
117                     date_counts[c] = date_counts.get(c, 0) + 1
118         if not date_counts:
119             return []
120         top = max(date_counts.values())
121         date_col = min(c for c, n in date_counts.items() if n == top)
122
123         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
124         if len(txn_rows) < 2:
125             return []
126
127         # A money column's money cells must outnumber its date cells ? so a sparsely-filled

```

```

deposit
128         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
129         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
130         money_cols = []
131         for c in range(ncols):
132             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
133             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
134             if m >= 1 and m > d:
135                 money_cols.append(c)
136         if len(money_cols) < 2:
137             return []
138         balance_col = max(money_cols)
139         flow_cols = [c for c in money_cols if c != balance_col]
140
141         text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
142         narr_col = (
143             max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
144             if text_cols
145             else None
146         )
147
148         out: list[ConceptRow] = []
149         prev = opening
150         for r in txn_rows:
151             bal = money(r[balance_col]) if balance_col < len(r) else None
152             if bal is None:
153                 return [] # a running balance on every row is required for the delta
154             amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
155             debit = credit = None
156             if amt is not None:
157                 if bal < prev - tol:
158                     debit = amt
159                 elif bal > prev + tol:
160                     credit = amt
161                 else:
162                     debit = amt # zero net with an amount present ? reconciliation will
flag it
163             out.append(
164                 ConceptRow(
165                     txn_date=r[date_col],
166                     narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
167                     debit=debit,
168                     credit=credit,
169                     balance=bal,
170                 )
171             )
172             prev = bal
173         return out
174
175
176     def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
177         """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.
178
179         Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
180         and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
181         reconciles, so the caller can refuse to persist.
182         """

```

```

183         with pdfplumber.open(pdf_path, password=password) as pdf:
184             words_by_page = [
185                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
186             ]
187             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
188             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
189
190             account_number = profile.first_account(head) or "UNKNOWN"
191             summary = parse_statement_summary(head)
192             oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)
193
194             confirmed: list[ConceptRow] = []
195             for table in tables:
196                 res = map_table(table, profile, oracle=oracle)
197                 if res and res.verified:
198                     confirmed.extend(res.rows)
199             if confirmed and oracle(confirmed):
200                 return SavingsParse(account_number, confirmed, True)
201
202             matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
203             clustered = map_table(matrix, profile, oracle=oracle)
204             if clustered and clustered.verified:
205                 return SavingsParse(account_number, clustered.rows, True)
206
207             # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
208             #     debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
209             #     direction from the running balance and let the oracle cross-check amount ==
|?balance|.
210             delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
211             if delta and oracle(delta):
212                 return SavingsParse(account_number, delta, True)
213
214             fallback = confirmed or (clustered.rows if clustered else [])
215             return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225

```

10. user (2026-06-10T16:13:53.510Z)

```

1     """HDFC savings / transaction-account statement parser.
2
3     **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4     default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5     table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6     decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7     position. pdfplumber's whole chain is permissive and it opens password-protected
8     statements directly.
9
10    **Layout.** Each transaction is one *primary* line ? the transaction date at the
11    far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12    Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13    narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14    cheque/reference number. We group words into visual lines by ``top``, detect

```

```

15     primary lines by a leading date + a closing-balance amount, attach continuations,
16     and read columns by x-position.
17
18     **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19     balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20     ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21     statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22     parse complete without inspecting any individual value ? which is how we validate
23     against real (sensitive) statements. No statement content is logged here.
24     """
25
26     from __future__ import annotations
27
28     import csv
29     import re
30     from dataclasses import dataclass, field
31     from pathlib import Path
32
33     import pdfplumber
34
35     from muneem import __version__
36     from muneem import db as dbmod
37     from muneem.errors import ParseError
38     from muneem.redaction import redact_filename
39
40     from . import register_parser
41     from .base import StatementParser
42     from .concepts import ConceptRow, StatementConcepts
43     from .profiles import HDFC_PROFILE
44     from .validation import (
45         BookendReport,
46         Confidence,
47         ReconcileReport,
48         reconcile_running_balance,
49         verify,
50     )
51
52     # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
53     # Left-aligned columns key on x0; right-aligned amount columns key on their right
54     # edge (x1), which is stable regardless of the number's magnitude.
55     _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
56     _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
57     _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
58     _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
59     _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
60     _LINE_TOL = 2.5 # words within this many points of each other share a visual line
61     _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
62
63     _DATE_RE = re.compile(r"^\d{2}/\d{2}/\d{4}$")
64     # Indian-format money: optional minus, comma groups, exactly two decimals, optional
65     Cr/Dr.
66     _AMOUNT_RE = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
67     _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
68
69     @dataclass
70     class Transaction:
71         date: str # dd/mm/yyyy, as printed
72         narration: str
73         withdrawal: float
74         deposit: float
75         closing_balance: float
76         value_date: str | None = None
77         ref: str | None = None
78         page: int = 0

```

```

79
80     def to_concept(self) -> ConceptRow:
81         """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
verifier's input).
82
83         Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
``page``
84         field is HDFC-internal provenance and has no concept equivalent.
85         """
86         return ConceptRow(
87             txn_date=self.date,
88             narration=self.narration,
89             debit=self.withdrawal,
90             credit=self.deposit,
91             balance=self.closing_balance,
92             value_date=self.value_date,
93             reference=self.ref,
94         )
95
96
97     @dataclass
98     class Statement:
99         path: Path
100         transactions: list[Transaction] = field(default_factory=list)
101         account_number: str | None = None
102
103
104     # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
vocabulary +
105     # verifier (the concept-core refactor). These back-compat names keep HDFC's public
surface stable:
106     # StatementSummary == the statement-level concepts a SUMMARY block supplies
107     # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
checks)
108     StatementSummary = StatementConcepts
109     ValidationReport = BookendReport
110
111
112     def _to_amount(token: str) -> float | None:
113         """Parse an HDFC money token to a float, or None if it is not money.
114
115         Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
116         is real information and the reconciliation math needs it.
117         """
118         if not _AMOUNT_RE.match(token):
119             return None
120         cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
121         try:
122             return float(cleaned)
123         except ValueError:
124             return None
125
126
127     def _group_lines(words: list[dict]) -> list[list[dict]]:
128         """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
129         lines: list[list[dict]] = []
130         anchor_top: float | None = None
131         for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
132             if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
133                 lines[-1].append(w)
134             else:
135                 lines.append([w])
136                 anchor_top = w["top"]
137         return [sorted(line, key=lambda w: w["x0"]) for line in lines]
138

```



```

139
140 def _line_top(line: list[dict]) -> float:
141     return min(w["top"] for w in line)
142
143
144 def _has_narration(line: list[dict]) -> bool:
145     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
146
147
148 def _has_amount(line: list[dict]) -> bool:
149     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
150
151
152 def _has_leading_date(line: list[dict]) -> bool:
153     first = line[0]
154     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
155
156
157 def _has_footer_marker(line: list[dict]) -> bool:
158     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
159
160
161 def _is_primary(line: list[dict]) -> bool:
162     """A transaction's first line: a date at the far left + a closing-balance amount.
163
164     Requiring the closing-balance amount guards against stray dates in the page
165     header / account-summary box (e.g. the statement period).
166     """
167     if not _has_leading_date(line):
168         return False
169     return any(
170         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
171         _AMOUNT_RE.match(w["text"])
172         for w in line
173     )
174
175 def _is_continuation(line: list[dict]) -> bool:
176     """A narration-wrap line: narration-band text, no leading date, no amounts."""
177     return _has_narration(line) and not _has_leading_date(line) and not
178     _has_amount(line)
179
180 def _parse_transaction(
181     primary: list[dict], continuations: list[list[dict]], page_no: int
182 ) -> Transaction:
183     withdrawal = deposit = 0.0
184     closing: float | None = None
185     narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
186
187     for line in (primary, *continuations):
188         for w in line:
189             x0, x1, text = w["x0"], w["x1"], w["text"]
190             if x0 >= _AMOUNT_MIN_X0:
191                 val = _to_amount(text)
192                 if val is None:
193                     continue
194                 if x1 < _WITHDRAWAL_MAX_X1:
195                     withdrawal = val
196                 elif x1 < _DEPOSIT_MAX_X1:
197                     deposit = val
198             else:
199                 closing = val
200             elif x0 >= _NARR_MIN_X0:
201                 narr.append((w["top"], x0, text))

```

```

202
203     value_date: str | None = None
204     parts: list[str] = []
205     for _top, _x0, text in sorted(narr):
206         if value_date is None and _DATE_RE.match(text):
207             value_date = text # the value date, lifted out of the narration band
208             continue
209             parts.append(text)
210
211     ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
212     ref = max(ref_candidates, key=len) if ref_candidates else None
213
214     return Transaction(
215         date=primary[0]["text"],
216         narration=" ".join(parts),
217         withdrawal=withdrawal,
218         deposit=deposit,
219         closing_balance=closing if closing is not None else 0.0,
220         value_date=value_date,
221         ref=ref,
222         page=page_no,
223     )
224
225
226     def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
227         """Pure, PDF-free core: turn grouped table-body lines into transactions.
228
229         Kept separate from PDF I/O so the column logic is unit-testable on synthetic
230         word lists, with no real data.
231         """
232         primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
233         transactions: list[Transaction] = []
234         for k, idx in enumerate(primaries):
235             stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
236             primary = lines[idx]
237             primary_top = _line_top(primary)
238             continuations: list[list[dict]] = []
239             for m in range(idx + 1, stop):
240                 line = lines[m]
241                 if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
242                     break
243                 if _is_continuation(line):
244                     continuations.append(line)
245             transactions.append(_parse_transaction(primary, continuations, page_no))
246         return transactions
247
248
249     def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
250         """Lines below the column header (drops the per-page account-summary box)."""
251         header_top: float | None = None
252         for line in lines:
253             texts = {w["text"] for w in line}
254             if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255                 header_top = _line_top(line)
256                 break
257         if header_top is None:
258             return []
259         return [line for line in lines if _line_top(line) > header_top + 1]
260
261
262     def _extract_account_number(text: str) -> str | None:
263         """Account number via the HDFC profile (bank knowledge as data; same pattern as
before)."""

```

```

264         return HDFC_PROFILE.first_account(text)
265
266
267     def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
268         """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
269
270         ``password`` is forwarded to pdfplumber for encrypted statements and used
271         in-memory only ? never stored or logged.
272         """
273         path = Path(pdf_path)
274         transactions: list[Transaction] = []
275         account_number: str | None = None
276         with pdfplumber.open(str(path), password=password or "") as pdf:
277             for page_no, page in enumerate(pdf.pages):
278                 if account_number is None and page_no < 2:
279                     account_number = _extract_account_number(page.extract_text() or "")
280                     words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
281                     body = _table_body(_group_lines(words))
282                     transactions.extend(extract_transactions_from_lines(body, page_no))
283             return Statement(path=path, transactions=transactions,
account_number=account_number)
284
285
286     def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
287         """Check the running balance via the shared reconciler (closing[i] ==
288         closing[i-1] - withdrawal + deposit).
289
290         Spans page breaks (transactions are in document order), so a clean report also
291         validates page stitching. Reports counts only ? no amounts.
292         """
293         entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
statement.transactions]
294         return reconcile_running_balance(entries, tol)
295
296
297     def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
298         """Pure core: read HDFC's SUMMARY block from extracted text lines.
299
300         Layout (stable across statements)::
301
302             SUMMARY
303             Opening Balance  Debit Amount  Credit Amount  Closing Balance
304             <opening>        <debitAmt>    <creditAmt>    <closing>
305             Debit Count  Credit Count
306             <debitCount> <creditCount>
307
308         The values land on their own lines, so we take the first post-SUMMARY line
309         carrying ?4 money tokens (the balances) and the first carrying ?2 bare
310         integers and no money (the counts).
311         """
312         start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
313         if start is None:
314             return None
315         balances: list[float] | None = None
316         counts: list[int] | None = None
317         for ln in lines[start + 1 : start + 8]:
318             tokens = ln.split()
319             monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
320             ints = [int(t) for t in tokens if t.isdigit()]
321             if balances is None and len(monies) >= 4:
322                 balances = monies[:4]
323             elif counts is None and not monies and len(ints) >= 2:
324                 counts = ints[:2]
325         if balances is None and counts is None:
326             return None

```

```

327     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328     opening, debit_amt, credit_amt, closing = bvals[:4]
329     cvals: list[int | None] = [*counts] if counts else [None, None]
330     debit_count, credit_count = cvals[:2]
331     return StatementSummary(
332         opening_balance=opening,
333         closing_balance=closing,
334         debit_count=debit_count,
335         credit_count=credit_count,
336         debit_amount=debit_amt,
337         credit_amount=credit_amt,
338     )
339
340
341 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344         for page in pdf.pages:
345             text = page.extract_text() or ""
346             if "SUMMARY" in text:
347                 summary = _parse_summary_lines(text.splitlines())
348                 if summary is not None:
349                     return summary
350     return None
351
352
353 def validate_against_summary(
354     statement: Statement, summary: StatementSummary, tol: float = 0.01
355 ) -> ValidationReport:
356     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357
358     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
361     closing
362     balances catch a missing first/last row.
363     """
364     rows = [t.to_concept() for t in statement.transactions]
365     return verify(rows, summary, tol)
366
367
368 def write_csv(statement: Statement, out_path: Path | str) -> Path:
369     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
370
371     Writes real data ? call only for a local, gitignored destination.
372     """
373     out = Path(out_path)
374     out.parent.mkdir(parents=True, exist_ok=True)
375     with open(out, "w", newline="", encoding="utf-8") as fh:
376         writer = csv.writer(fh)
377         writer.writerow(
378             [
379                 "date",
380                 "value_date",
381                 "narration",
382                 "ref",
383                 "withdrawal",
384                 "deposit",
385                 "closing_balance",
386                 "page",
387             ]
388         )
389         for t in statement.transactions:
390             writer.writerow(

```

```

391         t.date,
392         t.value_date or "",
393         t.narration,
394         t.ref or "",
395         f"{t.withdrawal:.2f}",
396         f"{t.deposit:.2f}",
397         f"{t.closing_balance:.2f}",
398         t.page,
399     ]
400 )
401 return out
402
403
404 @register_parser
405 class HDFCParser(StatementParser):
406     """Registered parser for HDFC savings / transaction-account statements."""
407
408     def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409         low = (text or "").lower()
410         is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"").lower()
411         is_statement = "savings account details" in low or (
412             "narration" in low and "closing balance" in low
413         )
414         return is_hdfc and is_statement
415
416     def parse_and_insert(
417         self,
418         pdf_path: Path,
419         conn,
420         doc_id: int,
421         password: str | None = None,
422     ) -> int:
423         """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425         Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426         balance walk (and the bank's SUMMARY when present) so a downstream consumer
427         can tell a trustworthy parse from a suspect one. Returns the row count.
428         """
429         try:
430             stmt = parse_statement(pdf_path, password=password or "")
431         except ParseError:
432             raise
433         except Exception as exc:
434             raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
435         for t in stmt.transactions:
436             dbmod.insert_hdfc_transaction(
437                 conn,
438                 document_id=doc_id,
439                 account_number=stmt.account_number,
440                 txn_date=t.date,
441                 value_date=t.value_date,
442                 particulars=t.narration,
443                 ref=t.ref,
444                 withdrawal=t.withdrawal,
445                 deposit=t.deposit,
446                 closing_balance=t.closing_balance,
447             )
448         summary = parse_summary(pdf_path, password=password or "")
449         ok = (
450             validate_against_summary(stmt, summary).ok
451             if summary is not None
452             else reconcile(stmt).ok

```

```

453         )
454         verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455         dbmod.record_parse(
456             conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457             parser_version=__version__
458         )
459         return len(stmt.transactions)

```

11. user (2026-06-10T16:13:54.048Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
4     table +
5     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
6     gated** so a
7     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
8     Axis specifics
9     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
10    mapping.
11
12    Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
13    reconcile
14    (parse/persist split + count only). No statement content is logged here.
15    """
16
17    from __future__ import annotations
18
19    import re
20    import sqlite3
21    from dataclasses import dataclass
22    from pathlib import Path
23
24    import pdfplumber
25
26    from muneem import __version__
27    from muneem import db as dbmod
28    from muneem.errors import ParseError
29    from muneem.redaction import redact_filename
30
31    from . import register_parser
32    from .base import StatementParser
33    from .profiles import AXIS_PROFILE
34    from .savings import SavingsParse, parse_savings, parse_statement_summary,
35    txn_line_predicate
36    from .validation import Confidence, parse_amount
37
38    _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
39    _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
40
41    def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
42        for rx in patterns:
43            m = rx.search(text)
44            if m:
45                return m.group(1).strip()
46        return None
47
48    # --- Savings / transaction account (concept core)
49    -----
50
51    # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with

```

```

AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(Axis_Profile)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, Axis_Profile, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66         def parse_and_insert(
67             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68         ) -> int:
69             """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71             Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72             parser can't yet read surfaces loudly instead of saving mislabeled rows.
73             """
74             try:
75                 parsed = parse_axis_savings(pdf_path, password)
76             except ParseError:
77                 raise
78             except Exception as exc:
79                 # Filename only ? never statement text; original cause kept via `from`.
80                 name = redact_filename(pdf_path)
81                 raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83             if not parsed.verified:
84                 dbmod.record_parse(
85                     conn,
86                     doc_id,
87                     confidence=Confidence.UNRECONCILED.value,
88                     parser="AxisParser",
89                     parser_version=__version__,
90                 )
91             return 0
92
93             for r in parsed.rows:
94                 dbmod.insert_axis_transaction(
95                     conn,
96                     document_id=doc_id,
97                     account_number=parsed.account_number,
98                     txn_date=r.txn_date,
99                     value_date=r.value_date or "",
100                     particulars=r.narration,
101                     debit=r.debit,
102                     credit=r.credit,
103                     balance=r.balance,
104                 )
105             dbmod.record_parse(
106                 conn,

```

```

107         doc_id,
108         confidence=Confidence.RECONCILED.value,
109         parser="AxisParser",
110         parser_version=__version__,
111     )
112     return len(parsed.rows)
113
114
115     # --- Credit card (no running balance; count only)
-----
116
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
130         for table in tables or []:
131             for row in table or []:
132                 if not row or len(row) < 9:
133                     continue
134                 date_str = str(row[0]).strip() if row[0] else ""
135                 if not _CC_DATE_RE.match(date_str):
136                     continue
137                 amount_str = str(row[8]).strip() if row[8] else ""
138                 is_cr = "Cr" in amount_str
139                 amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                 rows.append(
141                     AxisCCTxn(
142                         txn_date=date_str,
143                         particulars=str(row[2]).strip() if row[2] else "",
144                         merchant_category=str(row[7]).strip() if row[7] else "",
145                         amount=amount,
146                         type="Cr" if is_cr else "Dr",
147                     )
148                 )
149         return rows
150
151
152     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153         """Open an Axis credit-card statement and return (card_number, transactions)."""
154         card_number = "UNKNOWN"
155         rows: list[AxisCCTxn] = []
156         with pdfplumber.open(pdf_path, password=password) as pdf:
157             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159             for page in pdf.pages:
160                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text

```



```

168
169     def parse_and_insert(
170         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171     ) -> int:
172         try:
173             card_number, rows = parse_axis_cc(pdf_path, password)
174         except ParseError:
175             raise
176         except Exception as exc:
177             raise ParseError(
178                 f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179             ) from exc
180         for t in rows:
181             dbmod.insert_axis_cc_transaction(
182                 conn,
183                 document_id=doc_id,
184                 account_number=card_number,
185                 txn_date=t.txn_date,
186                 particulars=t.particulars,
187                 merchant_category=t.merchant_category,
188                 amount=t.amount,
189                 type=t.type,
190             )
191             # No running balance to reconcile; documents.status stays as the caller set it.
We still
192             # record provenance (with no confidence verdict).
193             dbmod.record_provenance(
194                 conn,
195                 document_id=doc_id,
196                 source_type="pdf_statement",
197                 parser="AxisCreditCardParser",
198                 parser_version=__version__,
199                 mapping_layer="deterministic",
200             )
201             return len(rows)
202

```

12. user (2026-06-10T16:13:59.612Z)

```

1     """AU Small Finance Bank savings / transaction-account statement parser.
2
3     Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
6     ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
here.
7     """
8
9     from __future__ import annotations
10
11     import sqlite3
12     from pathlib import Path
13
14     from muneem import __version__
15     from muneem import db as dbmod
16     from muneem.errors import ParseError
17     from muneem.redaction import redact_filename
18
19     from . import register_parser
20     from .base import StatementParser
21     from .profiles import AUBANK_PROFILE
22     from .savings import SavingsParse, parse_savings

```

```

23     from .validation import Confidence
24
25
26     def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
27         """Parse an AU Bank savings statement via the shared engine-driven path."""
28         return parse_savings(pdf_path, AUBANK_PROFILE, password)
29
30
31     @register_parser
32     class AUParser(StatementParser):
33         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
34             if sender and "aubank" in sender.lower():
35                 return True
36             low = (text or "").lower()
37             return "au small finance bank" in low or "aubank" in low
38
39         def parse_and_insert(
40             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
41         ) -> int:
42             """Persist a savings parse **only if it reconciles**; otherwise store
nothing."""
43             try:
44                 parsed = parse_au_savings(pdf_path, password)
45             except ParseError:
46                 raise
47             except Exception as exc:
48                 raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
49
50             if not parsed.verified:
51                 dbmod.record_parse(
52                     conn,
53                     doc_id,
54                     confidence=Confidence.UNRECONCILED.value,
55                     parser="AUParser",
56                     parser_version=__version__,
57                 )
58             return 0
59
60         for r in parsed.rows:
61             dbmod.insert_au_transaction(
62                 conn,
63                 document_id=doc_id,
64                 account_number=parsed.account_number,
65                 txn_date=r.txn_date,
66                 value_date=r.value_date or "",
67                 particulars=r.narration,
68                 debit=r.debit,
69                 credit=r.credit,
70                 balance=r.balance,
71             )
72             dbmod.record_parse(
73                 conn,
74                 doc_id,
75                 confidence=Confidence.RECONCILED.value,
76                 parser="AUParser",
77                 parser_version=__version__,
78             )
79         return len(parsed.rows)
80

```

```

1      from __future__ import annotations
2
3      import abc
4      import sqlite3
5      from pathlib import Path
6
7
8      def safe_amount(s: str) -> float | None:
9          """Parse a monetary string to float, or None for absent/zero cells.
10
11             ?? This maps ``"0.00"`` to ``None`` (treating a printed zero as *absent*). That is
fine for a
12             credit-card-style amount column but **wrong for balance-bearing rows**, where a real
``0.00``
13             balance must survive the reconciliation walk. For anything feeding reconciliation
use
14             ``muneem.parsers.validation.parse_amount`` instead (it keeps ``0.0`` distinct from
``None``).
15             """
16             if not s or s.strip() in ("-", "", "0.00"):
17                 return None
18             try:
19                 return float(s.replace(",", "").strip())
20             except ValueError:
21                 return None
22
23
24     class StatementParser(abc.ABC):
25         @abc.abstractmethod
26         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
27             """Return True if this parser can handle the given document."""
28             pass
29
30         @abc.abstractmethod
31         def parse_and_insert(
32             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
33         ) -> int:
34             """Extract txns and insert them into the bank-specific table. Returns count of
rows."""
35             pass
36
37
38     __all__ = ["StatementParser", "safe_amount"]
39

```

14. user (2026-06-10T16:14:00.941Z)

```

1      from __future__ import annotations
2
3      from .base import StatementParser, safe_amount
4
5      _REGISTRY: list[type[StatementParser]] = []
6
7
8      def register_parser(parser_cls: type[StatementParser]) -> type[StatementParser]:
9          """Decorator to register a parser."""
10         _REGISTRY.append(parser_cls)
11         return parser_cls
12
13
14     def get_parser(
15         text: str, sender: str | None = None, subject: str | None = None
16     ) -> StatementParser | None:

```

```

17         """Return an instantiated parser that can handle this document, or None."""
18         for parser_cls in _REGISTRY:
19             parser = parser_cls()
20             if parser.can_handle(text, sender, subject):
21                 return parser
22         return None
23
24
25 def _autoload_parsers() -> None:
26     """Import every parser submodule so its ``@register_parser`` runs.
27
28     Registration is an import *side-effect*, so the registry is only populated for
29     modules that have actually been imported. Without this, the production path
30     (``from muneem.parsers import get_parser``, as cli.py uses) would leave
31     ``_REGISTRY`` EMPTY ? only the tests imported the submodules. Auto-discovery
32     fixes that and means new parser modules register themselves with no edit here.
33     """
34     import importlib
35     import pkgutil
36
37     for info in pkgutil.iter_modules(__path__):
38         if info.name != "base" and not info.name.startswith("_"):
39             importlib.import_module(f"{__name__}.{info.name}")
40
41
42 _autoload_parsers()
43
44 __all__ = ["StatementParser", "register_parser", "get_parser", "safe_amount"]
45

```

15. user (2026-06-10T16:14:05.312Z)

```

1     from __future__ import annotations
2
3     import base64
4     import json
5     from pathlib import Path
6
7     from .base import EmailProvider, SearchCriteria
8
9     SCOPES = ["https://www.googleapis.com/auth/gmail.readonly"]
10    _TOKEN_KEY = "gmail_token"
11
12
13    class GmailProvider(EmailProvider):
14        def __init__(self, credentials_path: Path | str, open_browser: bool = True):
15            self.credentials_path = credentials_path
16            self.open_browser = open_browser
17            self.service = None
18
19        def authorize(self) -> None:
20            from google.auth.transport.requests import Request
21            from google.oauth2.credentials import Credentials
22            from google_auth_oauthlib.flow import InstalledAppFlow
23            from googleapiclient.discovery import build
24
25            from muneem import secrets
26
27            creds = None
28            raw = secrets.get_secret(_TOKEN_KEY)
29            if raw:
30                creds = Credentials.from_authorized_user_info(json.loads(raw), SCOPES)
31            if creds and creds.valid:
32                pass
33            elif creds and creds.expired and creds.refresh_token:

```

```

34         creds.refresh(Request())
35     else:
36         flow = InstalledAppFlow.from_client_secrets_file(str(self.credentials_path),
SCOPES)
37         creds = flow.run_local_server(port=0, open_browser=self.open_browser)
38     secrets.set_secret(_TOKEN_KEY, creds.to_json())
39     self.service = build("gmail", "v1", credentials=creds, cache_discovery=False)
40
41     def _build_query(self, criteria: SearchCriteria) -> str:
42         parts = []
43         if criteria.sender:
44             parts.append(f"from:{criteria.sender}")
45         if criteria.has_attachment:
46             parts.append("has:attachment")
47         if criteria.newer_than_days:
48             parts.append(f"newer_than:{criteria.newer_than_days}d")
49         return " ".join(parts)
50
51     def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
52         if not self.service:
53             self.authorize()
54
55         query = self._build_query(criteria)
56         ids: list[str] = []
57         page_token = None
58         while True:
59             resp = (
60                 self.service.users()
61                 .messages()
62                 .list(userId="me", q=query, pageToken=page_token,
maxResults=min(max_results, 100))
63                 .execute()
64             )
65             ids.extend(m["id"] for m in resp.get("messages", []))
66             page_token = resp.get("nextPageToken")
67             if not page_token or len(ids) >= max_results:
68                 break
69         return ids[:max_results]
70
71     def _get_message(self, message_id: str) -> dict:
72         if not self.service:
73             self.authorize()
74         return (
75             self.service.users().messages().get(userId="me", id=message_id,
format="full").execute()
76         )
77
78     def get_headers(self, message_id: str) -> dict[str, str]:
79         message = self._get_message(message_id)
80         return {
81             h.get("name", "").lower(): h.get("value", "")
82             for h in message.get("payload", {}).get("headers", [])
83         }
84
85     def _walk_parts(self, payload: dict):
86         stack = [payload]
87         while stack:
88             part = stack.pop()
89             yield part
90             stack.extend(part.get("parts", []))
91
92     def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
93         message = self._get_message(message_id)
94         out: list[tuple[str, str]] = []

```

```

95         for part in self._walk_parts(message.get("payload", {})):
96             filename = part.get("filename") or ""
97             att_id = part.get("body", {}).get("attachmentId")
98             if att_id and filename.lower().endswith(".pdf"):
99                 out.append((filename, att_id))
100         return out
101
102     def download_attachment(self, message_id: str, attachment_id: str, dest: Path) ->
Path:
103         if not self.service:
104             self.authorize()
105         att = (
106             self.service.users()
107             .messages()
108             .attachments()
109             .get(userId="me", messageId=message_id, id=attachment_id)
110             .execute()
111         )
112         dest = Path(dest)
113         dest.parent.mkdir(parents=True, exist_ok=True)
114         dest.write_bytes(base64.urlsafe_b64decode(att["data"].encode()))
115         return dest
116

```

16. user (2026-06-10T16:14:05.733Z)

```

1     from __future__ import annotations
2
3     from dataclasses import dataclass
4     from pathlib import Path
5     from typing import Protocol
6
7
8     @dataclass
9     class SearchCriteria:
10         sender: str | None
11         has_attachment: bool
12         newer_than_days: int | None
13
14
15     class EmailProvider(Protocol):
16         def authorize(self) -> None:
17             """Authenticate and authorize with the provider."""
18             ...
19
20         def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
21             """Return a list of message IDs matching the search criteria."""
22             ...
23
24         def get_headers(self, message_id: str) -> dict[str, str]:
25             """Return a dictionary of headers (keys lower-cased) for the message."""
26             ...
27
28         def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
29             """Return a list of (filename, attachment_id) for the message."""
30             ...
31
32         def download_attachment(self, message_id: str, attachment_id: str, dest: Path) ->
Path:
33             """Download the attachment to the specified path and return the path."""
34             ...
35

```

17. user (2026-06-10T16:14:12.594Z)

```

1      """Outlook / Microsoft Graph email provider ? ?? EXPERIMENTAL / INCOMPLETE.
2
3      Device-flow auth + basic search are implemented; attachment/header/download are
4      ``NotImplementedError``. This provider is **not wired into any CLI command** and is not
on a
5      supported path yet (see docs/CURRENT_STATE.md §1). Do not rely on it; it is excluded
from the
6      strict mypy gate (pyproject ``[[tool.mypy.overrides]]``) until the ``fetch`` command
lands
7      (ROADMAP B.4 / feedbackJune2026 P7). Requires the optional ``outlook`` extra (msal +
requests).
8      """
9
10     from __future__ import annotations
11
12     import json
13     from pathlib import Path
14
15     from .base import EmailProvider, SearchCriteria
16
17     _TOKEN_KEY = "outlook_token"
18     SCOPES = ["Mail.Read"]
19
20
21     class OutlookProvider(EmailProvider):
22         def __init__(self, client_id: str, authority: str =
"https://login.microsoftonline.com/common"):
23             self.client_id = client_id
24             self.authority = authority
25             self.access_token = None
26
27         def authorize(self) -> None:
28             try:
29                 from msal import PublicClientApplication
30             except ImportError:
31                 msg = "msal is required for OutlookProvider. Run pip install msal"
32                 raise RuntimeError(msg) from None
33
34             from muneem import secrets
35
36             app = PublicClientApplication(self.client_id, authority=self.authority)
37
38             # Try to find a cached token (simplified)
39             raw = secrets.get_secret(_TOKEN_KEY)
40             if raw:
41                 token_data = json.loads(raw)
42                 self.access_token = token_data.get("access_token")
43                 # Ideally, handle refresh logic here via msal TokenCache
44                 return
45
46             # Interactive login
47             flow = app.initiate_device_flow(scopes=SCOPES)
48             if "user_code" not in flow:
49                 raise ValueError("Failed to create device flow")
50
51             print(flow["message"]) # Instructs user to go to a URL and enter code
52             result = app.acquire_token_by_device_flow(flow)
53
54             if "access_token" in result:
55                 self.access_token = result["access_token"]
56                 secrets.set_secret(_TOKEN_KEY, json.dumps({"access_token":
self.access_token}))
57             else:
58                 raise RuntimeError(f"Failed to authenticate:
{result.get('error_description')}")

```

```

59
60     def _build_filter(self, criteria: SearchCriteria) -> str:
61         parts = []
62         if criteria.sender:
63             parts.append(f"from/emailAddress/address eq '{criteria.sender}'")
64         if criteria.has_attachment:
65             parts.append("hasAttachments eq true")
66         if criteria.newer_than_days:
67             pass # Graph API requires actual datetime subtraction for receivedDateTime
68
69         return " and ".join(parts)
70
71     def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
72         if not self.access_token:
73             self.authorize()
74
75         try:
76             import requests
77         except ImportError:
78             raise RuntimeError(
79                 "requests is required for OutlookProvider. Run: pip install requests
msal"
80             ) from None
81
82         url = "https://graph.microsoft.com/v1.0/me/messages"
83         headers = {"Authorization": f"Bearer {self.access_token}"}
84         params: dict = {"$select": "id"}
85
86         filter_str = self._build_filter(criteria)
87         if filter_str:
88             params["$filter"] = filter_str
89
90         params["$top"] = min(max_results, 100)
91
92         resp = requests.get(url, headers=headers, params=params)
93         resp.raise_for_status()
94
95         data = resp.json()
96         return [msg["id"] for msg in data.get("value", [])]
97
98     def get_headers(self, message_id: str) -> dict[str, str]:
99         raise NotImplementedError(
100             "OutlookProvider.get_headers is not yet implemented. "
101             "This provider is experimental. Only basic search works today."
102         )
103
104     def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
105         raise NotImplementedError(
106             "OutlookProvider.list_pdf_attachments is not yet implemented. "
107             "This provider is experimental. Only basic search works today."
108         )
109
110     def download_attachment(self, message_id: str, attachment_id: str, dest_dir: Path)
-> Path:
111         raise NotImplementedError(
112             "OutlookProvider.download_attachment is not yet implemented. "
113             "This provider is experimental. Only basic search works today."
114         )
115

```

18. user (2026-06-10T16:14:12.781Z)

```

1     from __future__ import annotations
2

```



```

3     from .base import EmailProvider, SearchCriteria
4     from .gmail import GmailProvider
5     from .imap import IMAPProvider
6     from .outlook import OutlookProvider
7
8     __all__ = ["EmailProvider", "SearchCriteria", "GmailProvider", "OutlookProvider",
9 "IMAPProvider"]

```

19. user (2026-06-10T16:14:14.554Z)

```

1     """IMAP email provider ? ?? EXPERIMENTAL / INCOMPLETE.
2
3     Basic auth + search are implemented; ``get_headers`` and the attachment methods are
4     ``NotImplementedError``. This provider is not wired into any CLI command** and is not
5     on a supported path yet (see docs/CURRENT_STATE.md §1). Do not rely on it; it is excluded
6     from the strict mypy gate (pyproject ``[[tool.mypy.overrides]]``) until the ``fetch`` command
7     lands (ROADMAP B.4 / feedbackJune2026 P7).
8     """
9
10    from __future__ import annotations
11
12    import imaplib
13    from pathlib import Path
14
15    from muneem import secrets
16
17    from .base import EmailProvider, SearchCriteria
18
19    _PASS_KEY = "imap_password"
20
21
22    class IMAPProvider(EmailProvider):
23        def __init__(self, host: str, username: str):
24            self.host = host
25            self.username = username
26            self.mail = None
27
28        def authorize(self) -> None:
29            password = secrets.get_secret(f"{_PASS_KEY}_{self.username}")
30            if not password:
31                import getpass
32
33                password = getpass.getpass(f"Enter App Password for {self.username} at
34 {self.host}: ")
35                secrets.set_secret(f"{_PASS_KEY}_{self.username}", password)
36
37            self.mail = imaplib.IMAP4_SSL(self.host)
38            self.mail.login(self.username, password)
39            self.mail.select("inbox", readonly=True)
40
41        def _build_search(self, criteria: SearchCriteria) -> list[str]:
42            search_args = []
43            if criteria.sender:
44                search_args.extend(["FROM", f'{criteria.sender}'])
45            # IMAP doesn't reliably support native "has attachment" searching natively on
46            all servers
47            # We can fetch structure later to filter, but we omit it from standard search
48            return search_args
49
50        def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
51 list[str]:

```

```

49         if not self.mail:
50             self.authorize()
51
52         search_args = self._build_search(criteria)
53         if not search_args:
54             search_args = ["ALL"]
55
56         status, messages = self.mail.search(None, *search_args)
57         if status != "OK":
58             raise RuntimeError("IMAP search failed")
59
60         msg_ids = messages[0].split()
61         return [mid.decode() for mid in msg_ids[-max_results:]]
62
63     def get_headers(self, message_id: str) -> dict[str, str]:
64         raise NotImplementedError(
65             "IMAPProvider.get_headers is not yet implemented. "
66             "Attachment download and header normalization are pending."
67         )
68
69     def list_pdf_attachments(self, message_id: str) -> list[tuple[str, str]]:
70         raise NotImplementedError(
71             "IMAPProvider.list_pdf_attachments is not yet implemented. "
72             "Attachment download and header normalization are pending."
73         )
74
75     def download_attachment(self, message_id: str, attachment_id: str, dest_dir: Path)
-> Path:
76         raise NotImplementedError(
77             "IMAPProvider.download_attachment is not yet implemented. "
78             "Attachment download and header normalization are pending."
79         )
80

```

20. user (2026-06-10T16:14:21.434Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")

```

```

30 app.add typer(offers_app, name="offers")
31
32 txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33 app.add typer(txns_app, name="txns")
34
35 db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36 app.add typer(db_app, name="db")
37
38
39 @contextmanager
40 def _ledger(db_path: Path):
41     """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
42     error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43     try:
44         with dbmod.connect_encrypted(db_path) as conn:
45             yield conn
46     except dbmod.EncryptedDatabaseError as exc:
47         typer.echo(str(exc), err=True)
48         raise typer.Exit(1) from exc
49
50
51 # Dispatch table for supported banks (extensible as new parsers land).
52 BANK_DISPATCH = {
53     "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54     "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55     "au": (dbmod.list_au_transactions, "debit", "credit"),
56     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57 }
58
59
60 def _version_callback(value: bool) -> None:
61     if value:
62         typer.echo(f"muneem {__version__}")
63         raise typer.Exit()
64
65
66 @app.callback()
67 def main(
68     version: bool = typer.Option(
69         False,
70         "--version",
71         callback=_version_callback,
72         is_eager=True,
73         help="Show the muneem version and exit.",
74     ),
75 ) -> None:
76     """muneem command-line interface."""
77
78
79 @app.command()
80 def info() -> None:
81     """Show resolved local paths (no secrets, no sensitive data)."""
82     cfg = load_config()
83     present = "present" if cfg.config_file.is_file() else "absent"
84     typer.echo(f"muneem {__version__}")
85     typer.echo(f"config dir : {cfg.config_dir}")
86     typer.echo(f"data dir   : {cfg.data_dir}")
87     typer.echo(f"db path    : {cfg.db_path}")
88     typer.echo(f"staging   : {cfg.staging_dir}")
89     typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92 def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:

```

```

93     """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95     The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
extraction,
96     since the text needed to pick a parser is itself encrypted. The resolved password is
forwarded
97     to extraction and the parser, used in-memory only ? never logged.
98     """
99     from muneem import db as dbmod
100    from muneem.extract import extract
101    from muneem.parsers import get_parser
102    from muneem.statement_unlock import resolve_password
103    from muneem.unlock import is_encrypted
104
105    password: str | None = ""
106    if is_encrypted(pdf):
107        password = resolve_password(pdf, interactive=interactive)
108        if password is None:
109            hint = " (run without --no-prompt to enter it)" if not interactive else ""
110            typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
111            raise typer.Exit(1)
112
113    ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
str
114    parser = get_parser(ex.text)
115    if parser is None:
116        typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
117        raise typer.Exit(1)
118
119    with _ledger(db_path) as conn:
120        dbmod.init_db(conn)
121        existing = dbmod.get_document_by_sha(conn, ex.sha256)
122        if existing is not None:
123            typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
124            return
125        doc_id = dbmod.insert_document(
126            conn,
127            source="file",
128            external_id=pdf.name,
129            doc_type=doc_type,
130            sha256=ex.sha256,
131            path=str(pdf.resolve()),
132            status="parsed",
133        )
134        count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
135        doc = dbmod.get_document_by_sha(conn, ex.sha256)
136        status = doc["status"] if doc else "parsed"
137
138        typer.echo(
139            f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
140            f"via {parser.__class__.__name__} [{status}]"
141        )
142
143
144    def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
145        """Run one statement through the full parse pipeline into an in-memory DB.
146
147        Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
148        (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
149        Nothing is persisted and no statement value is surfaced ? only the verdict and
counts.

```

```

150     """
151     from muneem import db as dbmod
152     from muneem.extract import extract
153     from muneem.parsers import get_parser
154     from muneem.statement_unlock import resolve_password
155     from muneem.unlock import is_encrypted
156
157     password = ""
158     if is_encrypted(pdf):
159         password = resolve_password(pdf, interactive=interactive) or ""
160         if not password:
161             return ("locked", 0, None)
162     try:
163         ex = extract(pdf, password=password)
164         parser = get_parser(ex.text)
165     except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
166         _log.warning(
167             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
168         )
169         return ("error", 0, None)
170     if parser is None:
171         return ("no-parser", 0, None)
172     name = parser.__class__.__name__
173     try:
174         with dbmod.connect(":memory:") as conn:
175             dbmod.init_db(conn)
176             doc_id = dbmod.insert_document(
177                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178             )
179             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180             doc = dbmod.get_document_by_sha(conn, ex.sha256)
181             status = doc["status"] if doc else "parsed"
182     except Exception as exc:
183         _log.warning(
184             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185         )
186         return ("error", 0, name)
187     return (status, count, name)
188
189
190 @app.command("eval")
191 def eval_statements(
192     folder: Path = typer.Argument(
193         ...,
194         exists=True,
195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",
202         help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203     ),
204 ) -> None:
205     """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207     The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few

```

```

*seed*
208     statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209     an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210     """
211     pdfs = sorted(folder.glob("*.pdf"))
212     if not pdfs:
213         typer.echo(f"no .pdf files in {folder}")
214         return
215     reconciled = 0
216     for pdf in pdfs:
217         outcome, count, name = _eval_statement(pdf, interactive=prompt)
218         reconciled += outcome == "reconciled"
219         typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220     typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223 @app.command()
224 def ingest(
225     pdf: Path = typer.Argument(
226         ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
227     ),
228     doc_type: str = typer.Option("promo", help="Document category."),
229     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230     ocr: bool = typer.Option(
231         False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232     ),
233     no_prompt: bool = typer.Option(
234         False,
235         "--no-prompt",
236         help="Never prompt for a statement password; use only cached/configured
candidates.",
237     ),
238     ) -> None:
239     """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240     from muneem.extract import extract
241     from muneem.offers import extract_offer
242
243     db_path = db or load_config().db_path
244
245     if doc_type == "bank_statement":
246         _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247         return
248
249     from muneem import db as dbmod
250
251     ex = extract(pdf)
252     text_fields = extract_offer(ex.text)
253     # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254     found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255     expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257     if ocr:
258         from muneem.ocr import ocr_pdf, tesseract_available
259
260         if not tesseract_available():
261             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262         else:
263             ocr_fields = extract_offer(ocr_pdf(pdf))
264             known = {c for c, _ in found}
265             for code in ocr_fields.codes:
266                 if code not in known:

```

```

267         found.append((code, "ocr"))
268         known.add(code)
269     if expiry_raw is None:
270         expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(
289                 conn,
290                 document_id=doc_id,
291                 promo_code=code,
292                 expiry_raw=expiry_raw,
293                 expiry_type=expiry_type,
294                 source=src,
295                 source_text=ex.text[:2000],
296             )
297
298     summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299     typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
305     dir)."),
306 ) -> None:
307     """List extracted offers."""
308     from muneem import db as dbmod
309
310     db_path = db or load_config().db_path
311     if not Path(db_path).exists():
312         typer.echo("no offers yet (database not created).")
313         return
314
315     with _ledger(db_path) as conn:
316         dbmod.init_db(conn)
317         rows = dbmod.list_offers(conn)
318
319     if not rows:
320         typer.echo("no offers found.")
321         return
322
323     for row in rows:
324         who = row["merchant"] or row["external_id"] or "?"
325         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
326         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
327         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
328
329
330 @txns_app.command("list")
331 def txns_list(
332     bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,

```

```

axis)."),
331         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332         limit: int = typer.Option(50, help="Max number of transactions to show."),
333     ) -> None:
334         """List parsed transactions for a specific bank."""
335         from muneem import db as dbmod
336
337         db_path = db or load_config().db_path
338         if not Path(db_path).exists():
339             typer.echo("database not created yet.")
340             return
341
342         bank = bank.lower()
343         if bank not in BANK_DISPATCH:
344             typer.echo(f"Unknown bank: {bank}", err=True)
345             return
346
347         list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348         with _ledger(db_path) as conn:
349             dbmod.init_db(conn)
350             rows = list_fn(conn, limit=limit)
351
352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:

```



```

391     """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392     from muneem import db as dbmod
393     from muneem.db_key import get_or_create_key
394
395     db_path = db or load_config().db_path
396     if not Path(db_path).exists():
397         typer.echo(
398             f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399         )
400         return
401     if not dbmod.is_plaintext_sqlite(db_path):
402         typer.echo(
403             f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404         )
405         return
406     n = dbmod.encrypt_database(db_path, get_or_create_key())
407     typer.echo(
408         f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409         f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410     )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()
422         if not key:
423             typer.echo("no encryption key in the keychain for this ledger.", err=True)
424             raise typer.Exit(1)
425         return key
426
427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
431     ) -> None:
432         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
433         from muneem import db as dbmod
434         from muneem import db_key
435
436         db_path = db or load_config().db_path
437         current = _require_encrypted_ledger(db_path)
438         fresh = db_key.new_key()
439         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
440         db_key.set_key(fresh)
441         try:
442             dbmod.rekey_database(db_path, current, fresh)
443         except dbmod.EncryptedDatabaseError as exc:
444             db_key.set_key(current) # roll back; the file is still under the old key
445             typer.echo(str(exc), err=True)
446             raise typer.Exit(1) from exc
447         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")

```

```

448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
500             "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501             "lost keychain means the ledger and its backups are unrecoverable."
502         )
503
504
505     @db_app.command("recover")

```

```

506     def db_recover(
507         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509     ) -> None:
510         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511         import getpass
512
513         from muneem import db as dbmod
514         from muneem import db_key, recovery
515
516         passphrase = getpass.getpass("Recovery passphrase: ")
517         try:
518             key = recovery.read_recovery_file(file, passphrase)
519         except recovery.RecoveryError as exc:
520             typer.echo(str(exc), err=True)
521             raise typer.Exit(1) from exc
522         db_path = db or load_config().db_path
523         # If the ledger is present, verify the recovered key actually opens it before
committing it.
524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

21. user (2026-06-10T16:14:26.812Z)

```

1     """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3     This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4     (accounts, transactions, holdings, merchants, items) grows with later milestones.
5     ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6     ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7     """
8
9     from __future__ import annotations
10
11     import os
12     import sqlite3
13     from collections.abc import Iterator
14     from contextlib import contextmanager
15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"

```

```

24
25 _SCHEMA = ""
26 CREATE TABLE IF NOT EXISTS documents (
27     id            INTEGER PRIMARY KEY,
28     source        TEXT NOT NULL,
29     external_id   TEXT,
30     sender        TEXT,
31     subject       TEXT,
32     received_date TEXT,
33     doc_type      TEXT,
34     sha256        TEXT NOT NULL UNIQUE,
35     path          TEXT,
36     status        TEXT NOT NULL DEFAULT 'parsed',
37     created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38 );
39
40 CREATE TABLE IF NOT EXISTS offers (
41     id            INTEGER PRIMARY KEY,
42     document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43     merchant      TEXT,
44     promo_code    TEXT,
45     description   TEXT,
46     expiry_raw    TEXT,
47     expiry_type   TEXT,
48     source        TEXT NOT NULL DEFAULT 'text',
49     redeemed      INTEGER NOT NULL DEFAULT 0,
50     source_text   TEXT
51 );
52
53 CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55 -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56 -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57 -- Normalization into a unified view layer is planned for a later phase
58 -- once all bank schemas are stabilized.
59 CREATE TABLE IF NOT EXISTS axis_transactions (
60     id INTEGER PRIMARY KEY,
61     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62     account_number TEXT,
63     txn_date TEXT,
64     value_date TEXT,
65     particulars TEXT,
66     debit REAL,
67     credit REAL,
68     balance REAL
69 );
70
71 CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72     id INTEGER PRIMARY KEY,
73     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74     account_number TEXT,
75     txn_date TEXT,
76     particulars TEXT,
77     merchant_category TEXT,
78     amount REAL,
79     type TEXT
80 );
81
82 CREATE TABLE IF NOT EXISTS au_transactions (
83     id INTEGER PRIMARY KEY,
84     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85     account_number TEXT,
86     txn_date TEXT,
87     value_date TEXT,
88     particulars TEXT,

```

```

89         debit REAL,
90         credit REAL,
91         balance REAL
92     );
93
94     CREATE TABLE IF NOT EXISTS hdfc_transactions (
95         id INTEGER PRIMARY KEY,
96         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97         account_number TEXT,
98         txn_date TEXT,
99         value_date TEXT,
100        particulars TEXT,
101        ref TEXT,
102        withdrawal REAL,
103        deposit REAL,
104        closing_balance REAL
105    );
106    """
107
108
109    def _raw_key_pragma(key: str) -> str:
110        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
111        KDF."""
112        return f"PRAGMA key = \"x'{key}'\""
113
114
115    def is_plaintext_sqlite(db_path: Path | str) -> bool:
116        """True iff the file exists and begins with the *unencrypted* SQLite magic header.
117
118        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
119        without needing the key.
120        """
121        try:
122            with open(db_path, "rb") as fh:
123                return fh.read(16) == _SQLITE_MAGIC
124        except OSError:
125            return False
126
127
128    def _harden_permissions(path: str) -> None:
129        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
130        ``:memory:``."""
131        if path == ":memory:":
132            return
133        try:
134            os.chmod(path, 0o600)
135        except OSError:
136            pass
137
138
139    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
140        """Open an encrypted connection via SQLCipher, verifying the key before returning.
141
142        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
143        the
144        key is wrong (or the file is not an encrypted muneem ledger), this raises.
145        """
146        try:
147            import sqlcipher3
148        except ImportError as exc: # pragma: no cover - exercised only without the binding
149            raise EncryptedDatabaseError(
150                "database encryption requested but the 'sqlcipher3' package is not
151                installed"
152            ) from exc

```

```

150     conn = sqlcipher3.connect(path)
151     conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152     try:
153         conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154     except sqlcipher3.DatabaseError as exc:
155         conn.close()
156         raise EncryptedDatabaseError(
157             f"could not open encrypted database {Path(path).name}: wrong key, or not an
158 "
159             "encrypted muneem ledger"
160         ) from exc
161     conn.row_factory = sqlcipher3.Row
162     return conn
163
164 @contextmanager
165 def connect(db_path: Path | str, *, key: str | None = None) ->
166     Iterator[sqlite3.Connection]:
167     """Open a connection (commit on success, always close). Parent dirs created.
168
169     With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
170     SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
171     path for
172     ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
173     :func:`_open_encrypted`); the production ledger is opened via
174     :func:`connect_encrypted`.
175
176     """
177     path = str(db_path)
178     if path != ":memory:":
179         Path(path).parent.mkdir(parents=True, exist_ok=True)
180     if key is not None:
181         conn = _open_encrypted(path, key)
182         _harden_permissions(path)
183     else:
184         conn = sqlite3.connect(path)
185         conn.row_factory = sqlite3.Row
186         conn.execute("PRAGMA foreign_keys = ON")
187     try:
188         yield conn
189         conn.commit()
190     finally:
191         conn.close()
192
193 @contextmanager
194 def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
195     """Open the real on-disk ledger, encrypted with the keychain-managed key.
196
197     The production path: resolves (or mints, on first use) the 256-bit key from the OS
198     keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
199     SQLite
200     file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
201     "file is not a database" error.
202
203     """
204     from muneem.db_key import get_or_create_key
205
206     path = str(db_path)
207     if is_plaintext_sqlite(path):
208         raise EncryptedDatabaseError(
209             f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
210 encrypt' "
211             "to migrate it to an encrypted ledger"
212         )
213     with connect(path, key=get_or_create_key()) as conn:
214         yield conn

```

```

209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
213         ``sqlcipher_export``.
214
215         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
216         then
217         securely delete (it still holds the cleartext data). Returns the number of user
218         tables
219         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
220         SQLite file.
221         """
222         import sqlcipher3
223
224         path = Path(db_path)
225         if not is_plaintext_sqlite(path):
226             raise EncryptedDatabaseError(
227                 f"{path.name} is not a plaintext SQLite database (already encrypted or
228                 missing)"
229             )
230
231         enc_tmp = path.with_name(path.name + ".enc.tmp")
232         enc_tmp.unlink(missing_ok=True)
233         try:
234             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
235             try:
236                 src_tables = src.execute(
237                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
238                 ).fetchone()[0]
239                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
240                 slashes
241                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY '{key}'")
242                 src.execute("SELECT sqlcipher_export('encrypted')")
243                 src.execute("DETACH DATABASE encrypted")
244             finally:
245                 src.close()
246
247         # Verify the encrypted copy BEFORE the destructive swap: it must open with the
248         key, pass
249         # an integrity check, and carry every table across. (The plaintext .bak is kept
250         either way,
251         # so a failed check is recoverable ? but we refuse to swap in a bad file.)
252         with connect(str(enc_tmp), key=key) as check:
253             integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
254             ntables = check.execute(
255                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
256             ).fetchone()[0]
257             if integrity != "ok" or ntables != src_tables:
258                 raise EncryptedDatabaseError(
259                     f"encryption migration verification failed for {path.name} "
260                     f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
261                     left untouched"
262                 )
263
264         backup = path.with_name(path.name + ".plaintext.bak")
265         backup.unlink(missing_ok=True)
266         path.rename(backup)
267         enc_tmp.rename(path) # consumed on success
268         _harden_permissions(str(path))
269         finally:
270             # Never leave a partial temp behind: no-op on success (already renamed), cleans
271             up on error.
272             enc_tmp.unlink(missing_ok=True)
273         return int(ntables)

```

```

264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = '{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318

```



```

319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
321                               (sha256,)).fetchone()
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
325         txns."""
326         cols = ", ".join(fields)
327         placeholders = ", ".join("?" for _ in fields)
328         cur = conn.execute(
329             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
330             tuple(fields.values()),
331         )
332         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
333         checker
334         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
335             raise RuntimeError(f"INSERT into {table} did not return a row id")
336         return int(rowid)
337
338     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
339         """Insert into documents table. Delegates to the shared helper for DRY."""
340         return _insert_row(conn, "documents", dict(fields))
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so
355         a single provenance record per document is reachable by join. Idempotent on ``document_id``
356         (a re-parse overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
360             "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361             "VALUES (?, ?, ?, ?, ?, ?) "
362             "ON CONFLICT(document_id) DO UPDATE SET "
363             "source_type=excluded.source_type, parser=excluded.parser, "
364             "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365             "confidence=excluded.confidence",
366             (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367         )
368
369     def record_parse(
370         conn: sqlite3.Connection,
371         document_id: int,
372         *,
373         confidence: str,
374         parser: str,
375         source_type: str = "pdf_statement",
376         parser_version: str | None = None,
377         mapping_layer: str = "deterministic",
378

```

```

379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
381         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
382         record_provenance(
383             conn,
384             document_id=document_id,
385             source_type=source_type,
386             parser=parser,
387             parser_version=parser_version,
388             mapping_layer=mapping_layer,
389             confidence=confidence,
390         )
391
392
393     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
394         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
395
396
397     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
398         return _insert_row(conn, "offers", fields)
399
400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405                    d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id
409             """
410         ).fetchall()
411
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:

```

```

438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

22. user (2026-06-10T16:14:32.550Z)

```

1     """PDF text extraction (the deterministic, born-digital path).
2
3     Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
4     extractor. We deliberately do *not* use PyMuPDF/`fitz`: it is **AGPL-3.0**,
5     incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).
6
7     NFKC normalization is applied because the real corpus returns ligatures
8     (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +
9     OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10    tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16    import unicodedata
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    from pdfminer.high_level import extract_text
21
22
23    @dataclass
24    class Extracted:
25        """The result of reading a PDF's text layer."""
26
27        path: Path
28        sha256: str
29        text: str
30
31
32    def _normalize(text: str) -> str:
33        """NFKC-normalize so ligatures fold to ASCII (U+FB00 '?' -> 'ff')."""
34        return unicodedata.normalize("NFKC", text)
35
36
37    def file_sha256(path: Path | str) -> str:
38        """Stream the file and return its hex SHA-256 (used for document dedup)."""
39        digest = hashlib.sha256()
40        with open(path, "rb") as fh:
41            for chunk in iter(lambda: fh.read(65536), b""):
42                digest.update(chunk)
43        return digest.hexdigest()
44
45
46    def extract(pdf_path: Path | str, password: str = "") -> Extracted:
47        """Extract and NFKC-normalize the full text layer of a PDF.

```

```

48
49     ``password`` is forwarded to pdfminer for encrypted statements and used in-memory
only ? it is
50     never stored or logged. The sha256 is over the file bytes, so it is independent of
the password.
51     """
52     path = Path(pdf_path)
53     text = extract_text(str(path), password=password or "")
54     return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))
55

```

23. user (2026-06-10T16:14:32.653Z)

```

1     """Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).
2
3     Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs full-
pipeline
4     fixtures ? real PDFs that go through pdfminer/pdfplumber, not synthetic word-lists that
bypass them.
5     Rather than add a dependency (reportlab etc.), we emit a minimal valid PDF with
positioned
6     Helvetica text** (`Tm`/`Tj`), so it stays commercial-safe and CI-portable.
7
8     Coordinates: callers think in pdfplumber's top-down `top` (distance from the page top);
we convert
9     to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key on
x-positions, same-row
10    grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
11    Amount columns are right-aligned via real Helvetica metrics so their right edges
(`x1`) land in
12    the bands the HDFC clusterer / engine expect.
13    """
14
15    from __future__ import annotations
16
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    # Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-
aligned amounts
21    # (digits / comma / period / minus / space); a reasonable default covers labels (left-
aligned, so
22    # their width is irrelevant to placement).
23    _W = {
24        " ": 278,
25        "!": 278,
26        "'": 355,
27        "#": 556,
28        "$": 556,
29        "%": 889,
30        "&": 667,
31        "'": 191,
32        "(": 333,
33        ")": 333,
34        "**": 389,
35        "+": 584,
36        ",": 278,
37        "-": 333,
38        ".": 278,
39        "/": 278,
40        "0": 556,
41        "1": 556,
42        "2": 556,
43        "3": 556,

```

```

44     "4": 556,
45     "5": 556,
46     "6": 556,
47     "7": 556,
48     "8": 556,
49     "9": 556,
50     ":" : 278,
51     "," : 278,
52     "<": 584,
53     "=": 584,
54     ">": 584,
55     "?": 556,
56     "@": 1015,
57 }
58 _UPPER = 667 # rough average for A-Z
59 _LOWER = 528 # rough average for a-z
60
61
62 def helvetica_width(text: str, size: float) -> float:
63     """Advance width of ``text`` in Helvetica at ``size`` points."""
64     total = 0
65     for ch in text:
66         if ch in _W:
67             total += _W[ch]
68         elif ch.isupper():
69             total += _UPPER
70         else:
71             total += _LOWER
72     return total / 1000.0 * size
73
74
75 @dataclass
76 class Placement:
77     """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""
78
79     x: float
80     top: float
81     text: str
82     size: float = 9.0
83
84
85 def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
86     """Left-aligned run starting at ``x``."""
87     return Placement(x=x, top=top, text=text, size=size)
88
89
90 def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
91     """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
92     return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)
93
94
95 def _esc(text: str) -> str:
96     return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
97
98
99 def build_pdf(
100     placements: list[Placement], path: Path | str, *, width: float = 595.0, height:
float = 842.0
101 ) -> Path:
102     """Write a single-page PDF with the given positioned text. Returns the path.
103
104     Hand-rolls a minimal PDF (catalog ? pages ? page ? Helvetica font ? content stream)
105     with a
106     byte-accurate xref table so pdfminer/pdfplumber parse it.
107     """

```

```

107 ops: list[str] = []
108 for p in placements:
109     y = height - p.top # top-down -> PDF bottom-up baseline
110     ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm
({_esc(p.text)}) Tj ET")
111     content = ("\n".join(ops)).encode("latin-1")
112
113     objects: list[bytes] = [
114         b"<< /Type /Catalog /Pages 2 0 R >>",
115         b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>",
116         (
117             f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
118             f"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
119         ).encode("latin-1"),
120         b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
121         b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream",
122     ]
123
124     out = bytearray(b"%PDF-1.4\n")
125     offsets: list[int] = []
126     for i, obj in enumerate(objects, start=1):
127         offsets.append(len(out))
128         out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"
129
130     xref_pos = len(out)
131     count = len(objects) + 1
132     out += f"xref\n0 {count}\n".encode()
133     out += b"0000000000 65535 f \n"
134     for off in offsets:
135         out += f"{off:010d} 00000 n \n".encode()
136     out += (
137         b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
138         b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
139     )
140
141     out_path = Path(path)
142     out_path.write_bytes(out)
143     return out_path
144
145
146 __all__ = ["Placement", "at", "right", "helvetica_width", "build_pdf"]
147

```

24. user (2026-06-10T16:14:37.294Z)

```

1     """Tests for the generic concept-mapping engine (muneem.parsers.engine).
2
3     Exercises the ladder on synthetic cell-matrices (no PDF/DB): L0 header lexicon, the
profile-order
4     fallback, L1/L2 content search with the oracle picking the self-identifying balance
column, and a
5     deliberately non-bank ``FamilySpec`` to prove the engine is family-agnostic.
6     """
7
8     import re
9     from dataclasses import dataclass
10
11     from muneem.parsers.engine import BANK_FAMILY, FamilySpec, Profile, map_table
12     from muneem.parsers.validation import parse_amount, reconcile_running_balance
13
14     DMY = (re.compile(r"^\d{2}/\d{2}/\d{4}$"),)
15     PROFILE = Profile(name="t", date_patterns=DMY)
16
17

```

```

18     def _walk(rows):
19         return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
20
21
22     # --- L0: header lexicon (generic labels, no custom aliases needed)
-----
23
24
25     def test_l0_header_lexicon_maps_generic_labels():
26         table = [
27             ["Date", "Particulars", "Withdrawals", "Deposits", "Balance"],
28             ["01/01/2026", "Opening Balance", "", "", "1000.00"], # marker -> skipped
29             ["02/01/2026", "UPI Amazon", "250.00", "", "750.00"],
30             ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
31         ]
32         r = map_table(table, PROFILE, BANK_FAMILY)
33         assert r is not None and r.layer == "L0"
34         assert [row.txn_date for row in r.rows] == ["02/01/2026", "03/01/2026"]
35         assert r.rows[0].debit == 250.0 and r.rows[0].balance == 750.0
36         assert r.rows[1].credit == 1250.0 and r.rows[1].balance == 2000.0
37         assert r.column_map["balance"] == 4
38
39
40     def test_l1_detects_value_date_as_second_date_column():
41         table = [
42             ["Tran Date", "Value Date", "Particulars", "Debit", "Credit", "Balance"],
43             ["02/01/2026", "01/01/2026", "UPI", "250.00", "", "750.00"],
44             ["03/01/2026", "02/01/2026", "Salary", "", "1250.00", "2000.00"],
45         ]
46         r = map_table(table, PROFILE, BANK_FAMILY)
47         assert r is not None
48         assert r.column_map["txn_date"] == 0 and r.column_map["value_date"] == 1
49         assert r.rows[0].value_date == "01/01/2026"
50
51
52     # --- profile-order: positional prior when there is no header
-----
53
54
55     def test_profile_order_fallback_when_no_header():
56         p = Profile(
57             name="t",
58             date_patterns=DMY,
59             column_order=("txn_date", "narration", "debit", "credit", "balance"),
60         )
61         table = [
62             ["02/01/2026", "UPI", "250.00", "", "750.00"],
63             ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
64         ]
65         r = map_table(table, p, BANK_FAMILY)
66         assert r is not None and r.layer == "profile-order"
67         assert r.rows[0].debit == 250.0 and r.rows[1].credit == 1250.0
68
69
70     # --- L2: the self-identifying balance column, found by the oracle, not by a label
-----
71
72
73     def test_l2_search_finds_self_identifying_balance_via_oracle():
74         # Unusual order, no header, no column_order: balance sits BEFORE the flows. Only the
assignment
75         # whose running balance walks cleanly should survive ? the oracle picks it.
76         table = [
77             ["02/01/2026", "UPI", "750.00", "250.00", ""], # date, narr, BALANCE, DEBIT,
CREDIT

```

```

78         ["03/01/2026", "Salary", "2000.00", "", "1250.00"],
79     ]
80     r = map_table(table, PROFILE, BANK_FAMILY, oracle=_walk)
81     assert r is not None and r.verified and r.layer == "L2"
82     assert r.column_map["balance"] == 2
83     assert r.rows[0].balance == 750.0 and r.rows[0].debit == 250.0
84     assert r.rows[1].credit == 1250.0 and r.rows[1].balance == 2000.0
85
86
87     def test_returns_unverified_when_nothing_reconciles():
88         # The Axis-style safety path: with an oracle, a non-reconciling parse comes back
verified=False
89         # so the caller can refuse to persist it.
90         table = [
91             ["02/01/2026", "UPI", "250.00", "", "750.00"],
92             ["03/01/2026", "Salary", "", "1250.00", "9999.00"], # balance broken
93         ]
94         r = map_table(table, PROFILE, BANK_FAMILY, oracle=_walk)
95         assert r is not None and not r.verified
96
97
98     def test_returns_none_when_no_transaction_rows():
99         table = [{"Date", "Particulars", "Balance"}, ["x", "y", "z"]]
100         assert map_table(table, PROFILE, BANK_FAMILY) is None
101
102
103     # --- family-agnostic seam: the same engine, a different concept family + oracle
-----
104
105
106     @dataclass
107     class PointsRow:
108         txn_date: str
109         earned: float | None
110         redeemed: float | None
111         balance: float | None
112
113
114     def _build_points(mapped):
115         return [
116             PointsRow(
117                 txn_date=m.get("txn_date", ""),
118                 earned=parse_amount(m.get("earned")),
119                 redeemed=parse_amount(m.get("redeemed")),
120                 balance=parse_amount(m.get("balance")),
121             )
122             for m in mapped
123         ]
124
125
126     POINTS_FAMILY = FamilySpec(
127         name="points",
128         concepts=("txn_date", "earned", "redeemed", "balance"),
129         date_concepts=("txn_date",),
130         amount_concepts=("earned", "redeemed", "balance"),
131         balance_concept="balance",
132         text_concepts=(),
133         default_aliases={
134             "txn_date": ("date",),
135             "earned": ("earned", "points in"),
136             "redeemed": ("redeemed", "points out"),
137             "balance": ("balance", "points balance"),
138         },
139         build=_build_points,
140     )

```



```

141
142
143 def _points_walk(rows):
144     prev, compared = None, 0
145     for r in rows:
146         if r.balance is None:
147             continue
148         if prev is not None:
149             compared += 1
150             if abs((prev + (r.earned or 0.0) - (r.redeemed or 0.0)) - r.balance) > 0.01:
151                 return False
152         prev = r.balance
153     return compared > 0
154
155
156 def test_family_agnostic_seam_drives_a_non_bank_family():
157     # No debit/credit/balance anywhere ? the engine maps a loyalty-points family with
its own
158     # concepts and reconciliation oracle, reusing the exact same L0/L1/L2 control flow.
159     table = [
160         ["02/01/2026", "100.00", "", "100.00"], # earned 100 -> balance 100
161         ["03/01/2026", "", "40.00", "60.00"], # redeemed 40 -> balance 60
162     ]
163     r = map_table(table, PROFILE, POINTS_FAMILY, oracle=_points_walk)
164     assert r is not None and r.verified
165     assert r.rows[0].earned == 100.0 and r.rows[0].balance == 100.0
166     assert r.rows[1].redeemed == 40.0 and r.rows[1].balance == 60.0
167

```

25. user (2026-06-10T16:14:41.665Z)

```

1     """Tests for the Axis parsers (savings via the concept core + credit card).
2
3     Savings runs end-to-end on synthetic data through a pdfplumber mock (extract_tables /
extract_text /
4     extract_words), so the per-table ? engine ? reconcile path is exercised without a real
PDF;
5     ``parse_and_insert``'s store-iff-reconciled gate is checked by monkeypatching that
function.
6     Real encrypted statements run under the ``local_statements`` marker
(test_local_axis.py).
7     """
8
9     from pathlib import Path
10    from unittest.mock import MagicMock, patch
11
12    import muneem.db as dbmod
13    from muneem.parsers import axis
14    from muneem.parsers.axis import AxisCreditCardParser, AxisParser, AxisSavings,
parse_axis_summary
15    from muneem.parsers.concepts import ConceptRow
16
17
18    def test_can_handle():
19        assert AxisParser().can_handle("x", sender="alerts@axisbank.com")
20        assert AxisParser().can_handle("Axis Bank Statement of Account")
21        assert AxisParser().can_handle("Axis Bank Account Statement") # the real wording on
statements
22        assert not AxisParser().can_handle("HDFC Bank")
23        assert AxisCreditCardParser().can_handle("Axis Bank Credit Card Statement")
24        assert not AxisCreditCardParser().can_handle("Axis Bank Statement of Account")
25
26
27    def test_parse_axis_summary_reads_opening_closing():
28        s = parse_axis_summary("Opening Balance 1,000.00\n...rows...\nClosing Balance

```

```

2,500.50")
29         assert s.opening_balance == 1000.0 and s.closing_balance == 2500.5
30
31
32     def test_is_txn_line():
33         assert axis._is_txn_line(["01-02-2026", "UPI/Zomato", "500.00", "", "9500.00"])
34         assert not axis._is_txn_line(["Tran Date", "Transaction", "Withdrawals", "Balance"])
# header
35         assert not axis._is_txn_line(["01-02-2026", "narration but no amount"]) # date, no
money
36
37
38     def _run_savings(page_tables, text="", page_words=None):
39         """Drive parse_axis_savings against a single synthetic page via a pdfplumber
mock."""
40         page = MagicMock()
41         page.extract_tables.return_value = page_tables
42         page.extract_text.return_value = text
43         page.extract_words.return_value = page_words or []
44         pdf = MagicMock()
45         pdf.pages = [page]
46         with patch("pdfplumber.open") as m:
47             m.return_value.__enter__.return_value = pdf
48             return axis.parse_axis_savings(Path("x.pdf"), password="")
49
50
51     def test_savings_per_row_reconcile_via_tables():
52         table = [
53             ["Tran Date", "Transaction", "Withdrawals", "Deposits", "Balance"],
54             ["01-02-2026", "UPI Zomato", "500.00", "", "9500.00"],
55             ["02-02-2026", "Salary", "", "2000.00", "11500.00"],
56             ["03-02-2026", "ATM", "1500.50", "", "9999.50"],
57         ]
58         r = _run_savings([table])
59         assert r.verified and len(r.rows) == 3
60         assert r.rows[0].debit == 500.0 and r.rows[0].balance == 9500.0
61         assert r.rows[1].credit == 2000.0
62
63
64     def test_savings_total_reconcile_when_no_per_row_balance():
65         # No Balance column ? reconciles only against the printed opening/closing totals
(Oct vintage).
66         table = [
67             ["Tran Date", "Transaction", "Withdrawals", "Deposits"],
68             ["01-02-2026", "UPI", "500.00", ""],
69             ["02-02-2026", "Salary", "", "2000.00"],
70         ]
71         r = _run_savings([table], text="Opening Balance 1,000.00 ... Closing Balance
2,500.00")
72         assert r.verified and len(r.rows) == 2
73         assert r.rows[0].debit == 500.0 and r.rows[0].balance is None
74
75
76     def test_savings_refuses_when_unreconciled():
77         table = [
78             ["Tran Date", "Transaction", "Withdrawals", "Deposits", "Balance"],
79             ["01-02-2026", "UPI", "500.00", "", "9500.00"],
80             ["02-02-2026", "Salary", "", "2000.00", "1.00"], # balance breaks the walk
81         ]
82         r = _run_savings([table])
83         assert not r.verified
84
85
86     def test_parse_and_insert_stores_when_verified(tmp_path, monkeypatch):
87         rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0,

```

```

balance=9500.0)]
88     monkeypatch.setattr(axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1",
rows, True))
89     with dbmod.connect(tmp_path / "t.db") as conn:
90         dbmod.init_db(conn)
91         doc_id = dbmod.insert_document(conn, source="file", sha256="ax1",
status="parsed")
92         n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
93         assert n == 1
94         stored = dbmod.list_axis_transactions(conn)
95         assert (
96             len(stored) == 1
97             and stored[0]["account_number"] == "ACC1"
98             and stored[0]["debit"] == 500.0
99         )
100         assert dbmod.get_document_by_sha(conn, "ax1")["status"] == "reconciled"
101
102
103     def test_parse_and_insert_refuses_when_unverified(tmp_path, monkeypatch):
104         rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0,
balance=1.0)]
105         monkeypatch.setattr(
106             axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1", rows, False)
107         )
108         with dbmod.connect(tmp_path / "t.db") as conn:
109             dbmod.init_db(conn)
110             doc_id = dbmod.insert_document(conn, source="file", sha256="ax2",
status="parsed")
111             n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
112             assert n == 0
113             assert dbmod.list_axis_transactions(conn) == []
114             assert dbmod.get_document_by_sha(conn, "ax2")["status"] == "unreconciled"
115
116
117     def test_rows_from_cc_tables():
118         tables = [
119             [
120                 ["Date", "c1", "Particulars", "c3", "c4", "c5", "c6", "Category", "Amount"],
121                 ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food", "250.00 Dr"],
122                 ["06/02/2026", "x", "Refund", "", "", "", "", "Food", "100.00 Cr"],
123                 ["bad", "too", "short"], # < 9 cols -> skipped
124             ]
125         ]
126         rows = axis._rows_from_cc_tables(tables)
127         assert len(rows) == 2
128         assert rows[0].amount == 250.0 and rows[0].type == "Dr" and
rows[0].merchant_category == "Food"
129         assert rows[1].amount == 100.0 and rows[1].type == "Cr"
130

```

26. user (2026-06-10T16:14:48.940Z)

```

1     """Tests for coordinate clustering (muneem.parsers.clustering).
2
3     Synthetic word geometry only ? proves lines group by ``top``, columns come from the
transaction
4     band (a full-width header must not collapse them), and a single wide cell can't merge
two columns.
5     """
6
7     from muneem.parsers.clustering import cluster_to_matrix, group_lines, infer_columns
8
9
10    def W(text, x0, top, x1=None):
11        """A minimal pdfplumber-style word dict."""

```

```

12     return {
13         "text": text,
14         "x0": float(x0),
15         "x1": float(x1 if x1 is not None else x0 + 5 * len(text)),
16         "top": float(top),
17         "bottom": float(top) + 8,
18     }
19
20
21 def _has_date(texts):
22     return any(t.count("-") == 2 for t in texts)
23
24
25 def _txn_row(date, narr, amt, bal, top):
26     return [
27         w(date, 40, top, 80),
28         w(narr, 110, top, 230),
29         w(amt, 300, top, 340),
30         w(bal, 420, top, 470),
31     ]
32
33
34 def test_group_lines_clusters_by_top():
35     words = [w("a", 10, 100), w("b", 60, 101), w("c", 10, 130)]
36     lines = group_lines(words)
37     assert len(lines) == 2
38     assert [w["text"] for w in lines[0]] == ["a", "b"]
39
40
41 def test_cluster_keeps_only_txn_lines_and_finds_columns():
42     words = [w("ACCOUNT STATEMENT", 40, 50, 520)] # full-width header ? dropped by
keep_line
43     words += _txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)
44     words += _txn_row("02-02-2026", "Salary", "2000.00", "11500.00", 120)
45     m = cluster_to_matrix([words], keep_line=_has_date)
46     assert len(m) == 2 # the header line is excluded
47     assert all(len(row) == 4 for row in m) # date | narration | amount | balance
48     assert m[0][0] == "01-02-2026" and m[0][3] == "9500.00"
49
50
51 def test_one_wide_cell_does_not_collapse_columns():
52     # A single long narration spanning toward the amount column must not merge the two
columns;
53     # the coverage histogram thresholds on the fraction of rows, so one outlier can't.
54     words = _txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)
55     words += _txn_row("02-02-2026", "Salary", "2000.00", "11500.00", 120)
56     words += [
57         w("01-02-2026", 40, 140, 80),
58         w("a very long narration line here", 110, 140, 295),
59         w("750.00", 300, 140, 340),
60         w("9000.00", 420, 140, 470),
61     ]
62     m = cluster_to_matrix([words], keep_line=_has_date)
63     assert len(m) == 3
64     assert all(len(row) == 4 for row in m)
65
66
67 def test_infer_columns_returns_runs():
68     lines = [_txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)]
69     cols = infer_columns(lines, min_frac=0.3)
70     assert len(cols) == 4
71

```

```

1      """Tests for the shared savings path's balance-delta reconstruction
(muneem.parsers.savings).
2
3      `_reconstruct_by_balance_delta` recovers debit/credit from the running-balance delta
when columns
4      don't split cleanly (a merged amount column, an interleaved value-date column, sparse
deposits). It
5      is exercised here on synthetic cell-matrices ? no PDF, no secrets ? including the
*dropped-row* case
6      (a balance jump matching no single amount) which must NOT reconcile (the real Axis May
2025 case).
7      """
8
9      from muneem.parsers import savings
10     from muneem.parsers.profiles import AXIS_PROFILE
11     from muneem.parsers.validation import reconcile_running_balance
12
13
14     def _matrix():
15         """June-style: date | narration | withdrawal(sparse) | deposit(sparse) | value-date
| balance.
16         Opening 10,000 ? 9,000 (?1,000) ? 14,000 (+5,000) ? 12,000 (?2,000)."""
17         return [
18             ["02-08-2025", "UPI-ABC", "1,000.00", "", "01-08-2025", "9,000.00"],
19             ["05-08-2025", "SALARY", "", "5,000.00", "04-08-2025", "14,000.00"],
20             ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "12,000.00"],
21         ]
22
23
24     def _reconciles(rows):
25         return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
26
27
28     def test_reconstructs_direction_from_balance_delta_and_reconciles():
29         rows = savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE,
opening=10000.0)
30         assert len(rows) == 3
31         assert _reconciles(rows)
32         # direction derived from the sign of ?balance; the value-date column is ignored as a
flow
33         assert rows[0].debit == 1000.0 and rows[0].credit is None
34         assert rows[1].credit == 5000.0 and rows[1].debit is None
35         assert rows[2].debit == 2000.0 and rows[2].credit is None
36         assert [r.balance for r in rows] == [9000.0, 14000.0, 12000.0]
37
38
39     def test_a_dropped_transaction_does_not_reconcile():
40         # Drop a transaction (make the last balance jump): |?balance| no longer equals the
row's amount,
41         # so the reconstruction must NOT reconcile ? the oracle quarantines an incomplete
ledger (May).
42         m = _matrix()
43         m[2] = ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "1,000.00"]
44         rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
45         assert not _reconciles(rows)
46
47
48     def test_requires_opening_balance_to_fix_first_direction():
49         assert savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE, opening=None)
== []
50
51
52     def test_ignores_a_leading_non_transaction_noise_row():
53         # A header/summary line (no leading date) must be dropped, not parsed as a
transaction.

```

```

54     m = [["Opening Balance", "", "", "", "", "10,000.00"], *_matrix()]
55     rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
56     assert len(rows) == 3 and _reconciles(rows)
57

```

28. user (2026-06-10T16:14:55.184Z)

```

1     """Tests for the verifier (muneem.parsers.validation): the per-row walk, the layout-
independent
2     total reconcile, the concept-level bookend, candidate selection, and the confidence
verdict."""
3
4     from muneem.parsers.concepts import ConceptRow, StatementConcepts
5     from muneem.parsers.validation import (
6         Confidence,
7         reconcile_running_balance,
8         reconcile_total,
9         select_reconciling,
10        verify,
11    )
12
13
14    def _concept_rows():
15        """3 rows that walk cleanly: 10000 -500-> 9500 +2000-> 11500 -1500.5-> 9999.5."""
16        return [
17            ConceptRow(txn_date="05/02/2026", narration="a", debit=500.0, credit=0.0,
balance=9500.0),
18            ConceptRow(txn_date="08/02/2026", narration="b", debit=0.0, credit=2000.0,
balance=11500.0),
19            ConceptRow(txn_date="10/02/2026", narration="c", debit=1500.5, credit=0.0,
balance=9999.5),
20        ]
21
22
23    def test_reconciles_clean():
24        entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.5)]
25        r = reconcile_running_balance(entries)
26        assert r.n == 3 and r.compared == 2 and r.breaks == 0 and r.ok
27
28
29    def test_detects_break():
30        entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.49)]
31        r = reconcile_running_balance(entries)
32        assert r.breaks == 1 and r.first_break == 2 and not r.ok
33
34
35    def test_none_debit_or_credit_treated_as_zero():
36        # Bank rows: exactly one of (debit, credit) is filled; the other is None.
37        entries = [(None, None, 1000.0), (250.0, None, 750.0), (None, 1250.0, 2000.0)]
38        r = reconcile_running_balance(entries)
39        assert r.compared == 2 and r.breaks == 0 and r.ok
40
41
42    def test_a_dropped_balance_surfaces_as_a_gap():
43        # If a row's balance goes missing (e.g. a silent extract_tables drop), the walk
44        # across the gap no longer adds up ? exactly the failure we want surfaced.
45        entries = [(None, None, 1000.0), (250.0, None, None), (None, None, 750.0)]
46        r = reconcile_running_balance(entries)
47        assert r.breaks == 1 and r.first_break == 2 and not r.ok
48
49
50    def test_empty_and_single_are_not_ok():
51        assert not reconcile_running_balance([]).ok
52        assert not reconcile_running_balance([(None, None, 100.0)]).ok # nothing to compare
53

```

```

54
55     # --- total reconcile: opening + ?credit ? ?debit == closing (layout-independent)
56     -----
57
58     def test_reconcile_total_clean_and_broken():
59         rows = _concept_rows()
60         assert reconcile_total(rows, opening=10000.0, closing=9999.5)
61         assert not reconcile_total(rows, opening=10000.0, closing=9990.0)
62
63
64     def test_reconcile_total_needs_no_per_row_balance():
65         # Credit-card-style rows (balance=None) still validate against the opening/closing
66         bookends.
67         rows = [
68             ConceptRow(txn_date="01/03/2026", narration="charge", debit=300.0),
69             ConceptRow(txn_date="02/03/2026", narration="payment", credit=100.0),
70         ]
71         assert reconcile_total(rows, opening=1000.0, closing=800.0) # 1000 ? 300 + 100
72         assert not reconcile_total(rows, opening=1000.0, closing=999.0)
73
74     # --- bookend (verify) over the concept schema
75     -----
76
77     def test_verify_full_bookend_passes_and_reports_confidence():
78         concepts = StatementConcepts(
79             opening_balance=10000.0, closing_balance=9999.5, debit_count=2, credit_count=1
80         )
81         r = verify(_concept_rows(), concepts)
82         assert r.reconciles and r.ok
83         assert r.debit_count_ok and r.credit_count_ok and r.opening_ok and r.closing_ok
84         assert r.confidence is Confidence.RECONCILED
85
86
87     def test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
88         # No SUMMARY figures: the per-row walk still passes, but with nothing to bookend
89         against the
90         # parse is not *trusted* (ok=False). This is why HDFC falls back to reconcile() when
91         absent.
92         r = verify(_concept_rows(), StatementConcepts())
93         assert r.reconciles
94         assert r.debit_count_ok is None and r.credit_count_ok is None
95         assert r.opening_ok is None and r.closing_ok is None
96         assert not r.ok and r.confidence is Confidence.UNRECONCILED
97
98
99     def test_verify_catches_wrong_count():
100         concepts = StatementConcepts(
101             opening_balance=10000.0, closing_balance=9999.5, debit_count=1, credit_count=1
102         )
103         r = verify(_concept_rows(), concepts) # there are 2 debits, not 1
104         assert r.reconciles and r.debit_count_ok is False and not r.ok
105
106     # --- selection: arithmetic picks the right candidate mapping
107     -----
108
109     def test_select_reconciling_picks_the_arithmetic_winner():
110         good = _concept_rows()
111         swapped = [ # row 2's deposit mislabeled as a withdrawal ? the walk breaks
112             ConceptRow(txn_date="05/02/2026", narration="a", debit=500.0, credit=0.0,
113                 balance=9500.0),

```

```

112         ConceptRow(txn_date="08/02/2026", narration="b", debit=2000.0, credit=0.0,
balance=11500.0),
113         ConceptRow(txn_date="10/02/2026", narration="c", debit=1500.5, credit=0.0,
balance=9999.5),
114     ]
115     assert select_reconciling([swapped, good]) == 1
116     assert select_reconciling([swapped]) is None
117     assert select_reconciling([]) is None
118

```

29. user (2026-06-10T16:14:55.590Z)

```

1     """Property-style fuzzing of the reconciliation verifier ? deterministic (seeded), no
new dep.
2
3     The verifier is muneem's truth oracle: an off-by-one in the balance walk or the total
identity
4     would silently let mislabeled rows through. These tests generate many random
*consistent* ledgers
5     and assert the oracles accept them, then perturb a single value past the tolerance and
assert the
6     oracles catch it. A one-line slip in the reconcile arithmetic fails here.
7     """
8
9     import random
10
11     import pytest
12
13     from muneem.parsers.concepts import ConceptRow
14     from muneem.parsers.validation import reconcile_running_balance, reconcile_total
15
16
17     def _consistent_ledger(rng: random.Random, n: int, opening: float):
18         """Build a random clean ledger of `n` rows; return (rows, closing_balance)."""
19         rows: list[ConceptRow] = []
20         bal = opening
21         for _ in range(n):
22             amt = round(rng.uniform(0.01, 5000.0), 2)
23             if rng.random() < 0.5:
24                 bal = round(bal - amt, 2)
25                 rows.append(ConceptRow(txn_date="01/01/2026", narration="x", debit=amt,
balance=bal))
26             else:
27                 bal = round(bal + amt, 2)
28                 rows.append(ConceptRow(txn_date="01/01/2026", narration="x", credit=amt,
balance=bal))
29         return rows, bal
30
31
32     def _entries(rows):
33         return [(r.debit, r.credit, r.balance) for r in rows]
34
35
36     @pytest.mark.parametrize("seed", range(60))
37     def test_consistent_ledger_always_reconciles(seed):
38         rng = random.Random(seed)
39         opening = round(rng.uniform(0, 100_000), 2)
40         rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
41         assert reconcile_running_balance(_entries(rows)).ok
42         assert reconcile_total(rows, opening=opening, closing=closing)
43
44
45     @pytest.mark.parametrize("seed", range(60))
46     def test_single_balance_perturbation_is_caught_by_the_walk(seed):
47         rng = random.Random(10_000 + seed)

```



```

48     opening = round(rng.uniform(0, 100_000), 2)
49     rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
50     i = rng.randrange(len(rows))
51     r = rows[i]
52     rows[i] = ConceptRow( # corrupt one balance by more than the tolerance
53         txn_date=r.txn_date,
54         narration=r.narration,
55         debit=r.debit,
56         credit=r.credit,
57         balance=round((r.balance or 0.0) + 1.0, 2),
58     )
59     assert not reconcile_running_balance(_entries(rows)).ok
60
61
62     @pytest.mark.parametrize("seed", range(60))
63     def test_total_oracle_catches_a_wrong_closing_bookend(seed):
64         rng = random.Random(20_000 + seed)
65         opening = round(rng.uniform(0, 100_000), 2)
66         rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
67         assert reconcile_total(rows, opening=opening, closing=closing)
68         assert not reconcile_total(rows, opening=opening, closing=round(closing + 1.0, 2))
69

```

30. user (2026-06-10T16:15:02.641Z)

```

1     """Tests for the HDFC statement parser (muneem.parsers.hdfc).
2
3     Column logic is exercised on *synthetic* word lists that mimic the real statement
4     geometry ? no real data, fast, offline. ``parse_and_insert`` is tested by
5     monkeypatching the parse functions (not by mocking pdfplumber), and a
6     ``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
7     """
8
9     import os
10    from pathlib import Path
11
12    import pytest
13
14    from muneem import db as dbmod
15    from muneem.parsers import hdfc
16
17
18    def W(text, x0, top, x1=None):
19        """A minimal pdfplumber-style word dict."""
20        return {
21            "text": text,
22            "x0": float(x0),
23            "x1": float(x1 if x1 is not None else x0 + 5 * len(text)),
24            "top": float(top),
25            "bottom": float(top) + 8,
26        }
27
28
29    def _sample_words():
30        """A 3-transaction table whose running balance reconciles, with header, a
31        per-page summary box above it, wrapped narration, and a footer marker."""
32        words = []
33        words += [W("Opening", 37, 50), W("Balance", 72, 50), W("01/02/2026", 52, 60)]
34        words += [
35            W("Date", 72, 80),
36            W("Narration", 175, 80),
37            W("Withdrawals", 296, 80),
38            W("Deposits", 386, 80),
39            W("Closing", 473, 80),
40            W("Balance", 504, 80),

```

```

41     ]
42     words += [
43         W("05/02/2026", 52, 100, 92),
44         W("UPI-SWIGGY-PAY", 111, 100),
45         W("500.00", 330, 100, 358),
46         W("0.00", 428, 100, 443),
47         W("9,500.00", 519, 100, 561),
48     ]
49     words += [W("okhdfcbank-CONTINUED", 111, 109)]
50     words += [W("04/02/2026", 122, 118, 162), W("123456789012", 180, 118, 255)]
51     words += [
52         W("08/02/2026", 52, 140, 92),
53         W("SALARY-CREDIT", 111, 140),
54         W("0.00", 340, 140, 358),
55         W("2,000.00", 415, 140, 443),
56         W("11,500.00", 515, 140, 561),
57     ]
58     words += [
59         W("10/02/2026", 52, 160, 92),
60         W("ATM-WDL", 111, 160),
61         W("1,500.50", 320, 160, 358),
62         W("0.00", 428, 160, 443),
63         W("9,999.50", 519, 160, 561),
64     ]
65     words += [W("STATEMENT", 200, 180), W("SUMMARY", 260, 180)]
66     return words
67
68
69 def _parse_sample():
70     body = hdfc._table_body(hdfc._group_lines(_sample_words()))
71     return hdfc.extract_transactions_from_lines(body, page_no=3)
72
73
74 def _sample_statement():
75     return hdfc.Statement(
76         path=Path("x"), transactions=_parse_sample(), account_number="50100000000000"
77     )
78
79
80 def test_columns_and_continuation_assembly():
81     t1, t2, t3 = _parse_sample()
82     assert t1.date == "05/02/2026"
83     assert t1.withdrawal == 500.0 and t1.deposit == 0.0 and t1.closing_balance == 9500.0
84     assert t1.value_date == "04/02/2026"
85     assert t1.ref == "123456789012"
86     assert t1.narration == "UPI-SWIGGY-PAY okhdfcbank-CONTINUED 123456789012"
87     assert t1.page == 3
88     assert t2.deposit == 2000.0 and t2.closing_balance == 11500.0
89     assert t3.withdrawal == 1500.5 and t3.closing_balance == 9999.5
90
91
92 def test_summary_box_and_footer_excluded():
93     txns = _parse_sample()
94     assert [t.date for t in txns] == ["05/02/2026", "08/02/2026", "10/02/2026"]
95     assert "STATEMENT" not in txns[-1].narration and "SUMMARY" not in txns[-1].narration
96
97
98 def test_reconciliation():
99     assert hdfc.reconcile(_sample_statement()).ok
100     txns = _parse_sample()
101     txns[2].closing_balance = 9999.49 # off by a paisa
102     report = hdfc.reconcile(hdfc.Statement(path=Path("x"), transactions=txns))
103     assert report.breaks == 1 and report.first_break == 2 and not report.ok
104
105

```

```

106 def test_parse_summary_lines_and_validation():
107     lines = [
108         "SUMMARY",
109         "Opening Balance Debit Amount Credit Amount Closing Balance",
110         "10,000.00 2,000.50 2,000.00 9,999.50",
111         "Debit Count Credit Count",
112         "2 1",
113     ]
114     s = hdfc._parse_summary_lines(lines)
115     assert s.opening_balance == 10000.0 and s.closing_balance == 9999.5
116     assert s.debit_count == 2 and s.credit_count == 1
117     report = hdfc.validate_against_summary(_sample_statement(), s)
118     assert report.ok and report.debit_count_ok and report.credit_count_ok
119     assert report.opening_ok and report.closing_ok
120
121
122 def test_validation_catches_missing_last_row():
123     stmt = hdfc.Statement(path=Path("x"), transactions=_parse_sample()[:-1])
124     summary = hdfc.StatementSummary(
125         opening_balance=10000.0, closing_balance=9999.50, debit_count=2, credit_count=1
126     )
127     report = hdfc.validate_against_summary(stmt, summary)
128     assert report.reconciles and report.closing_ok is False and not report.ok
129
130
131 @pytest.mark.parametrize(
132     "token,expected",
133     [
134         ("1,23,456.78", 123456.78),
135         ("0.00", 0.0),
136         ("1,000.00Cr", 1000.0),
137         ("9,999.50Dr", 9999.50),
138         ("abc", None),
139         ("100", None),
140         ("10/02/2026", None),
141     ],
142 )
143 def test_to_amount(token, expected):
144     assert hdfc._to_amount(token) == expected
145
146
147 def test_transaction_to_concept_maps_onto_canonical_vocabulary():
148     """The reference parser maps its native row onto the bank-agnostic concept
149     schema."""
150     t = hdfc.Transaction(
151         date="05/02/2026",
152         narration="UPI-SWIGGY",
153         withdrawal=500.0,
154         deposit=0.0,
155         closing_balance=9500.0,
156         value_date="04/02/2026",
157         ref="123456789012",
158         page=3,
159     )
160     c = t.to_concept()
161     assert c.txn_date == "05/02/2026" and c.narration == "UPI-SWIGGY"
162     assert c.debit == 500.0 and c.credit == 0.0 and c.balance == 9500.0
163     assert c.value_date == "04/02/2026" and c.reference == "123456789012"
164
165 def test_can_handle():
166     parser = hdfc.HDFCParser()
167     statement = "HDFC BANK Savings Account Details Narration Closing Balance"
168     assert parser.can_handle(statement)
169     assert parser.can_handle(statement, sender="alerts@hdfcbank.net")

```

```

170     # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
171     assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
172     assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
173     assert not parser.can_handle("Axis Bank Statement of Account Narration Closing
Balance")
174
175
176     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
177         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
178         monkeypatch.setattr(
179             hdfc,
180             "parse_summary",
181             lambda *a, **k: hdfc.StatementSummary(
182                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
credit_count=1
183             ),
184         )
185         with dbmod.connect(tmp_path / "t.db") as conn:
186             dbmod.init_db(conn)
187             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
status="parsed")
188             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
189             assert n == 3
190             rows = dbmod.list_hdfc_transactions(conn, limit=100)
191             assert len(rows) == 3
192             assert rows[0]["account_number"] == "501000000000000"
193             assert {r["particulars"] for r in rows} # narration mapped to particulars
194             assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195
196
197     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198         broken = _parse_sample()
199         broken[2].closing_balance = 1.0 # break the balance walk
200         monkeypatch.setattr(
201             hdfc,
202             "parse_statement",
203             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204         )
205         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None) # fall back to
reconcile()
206         with dbmod.connect(tmp_path / "t.db") as conn:
207             dbmod.init_db(conn)
208             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
status="parsed")
209             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
210             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211
212
213     @pytest.mark.local_statements
214     @pytest.mark.parametrize(
215         "pdf_path",
216         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
217         ids=lambda p: p.name,
218     )
219     def test_real_hdfc_statement_validates(pdf_path, tmp_path):
220         """Local-only: parse the real statements and require the parse to agree with
HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and reconcile fully.
221
222         Set MUNEEM_TEST_HDFC_PW; never runs in CI. Surfaces counts/booleans only.
223         """
224         pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
225         if not pw:
226             pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
227
228         stmt = hdfc.parse_statement(pdf_path, password=pw)

```

```

230     assert stmt.transactions, f"{pdf_path.name}: no transactions parsed"
231     summary = hdfc.parse_summary(pdf_path, password=pw)
232     assert summary is not None, f"{pdf_path.name}: SUMMARY block not found"
233     report = hdfc.validate_against_summary(stmt, summary)
234     assert report.ok, (
235         f"{pdf_path.name}: parse disagrees with bank SUMMARY ?
reconciles={report.reconciles} "
236         f"dr={report.debit_count_ok} cr={report.credit_count_ok} "
237         f"open={report.opening_ok} close={report.closing_ok}"
238     )
239
240     # End-to-end through the registered parser + DB, incl. the status it records.
241     with dbmod.connect(tmp_path / "t.db") as conn:
242         dbmod.init_db(conn)
243         doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
244         n = hdfc.HDFCParser().parse_and_insert(pdf_path, conn, doc_id, password=pw)
245         assert n == len(stmt.transactions)
246         assert dbmod.get_document_by_sha(conn, pdf_path.name)["status"] == "reconciled"
247

```

31. user (2026-06-10T16:15:07.619Z)

```

1     """Tests for the AU Bank parser.
2
3     AU runs on the shared engine-driven savings path (covered in depth by test_axis /
test_engine), so
4     here we check AU detection, an end-to-end per-row reconcile via a pdfplumber mock, and
the
5     store-iff-reconciled gate. Real statements run under local_statements
(test_local_au.py).
6     """
7
8     from pathlib import Path
9     from unittest.mock import MagicMock, patch
10
11     import muneem.db as dbmod
12     from muneem.parsers import au
13     from muneem.parsers.au import AUParser, parse_au_savings
14     from muneem.parsers.concepts import ConceptRow
15     from muneem.parsers.savings import SavingsParse
16
17
18     def test_can_handle():
19         assert AUParser().can_handle("...customercare@aubank.in...")
20         assert AUParser().can_handle("AU Small Finance Bank Statement")
21         assert AUParser().can_handle("anything", sender="statements@aubank.in")
22         assert not AUParser().can_handle("HDFC Bank savings account details")
23         assert not AUParser().can_handle("Axis Bank Statement of Account")
24
25
26     def test_parse_au_savings_per_row_reconcile():
27         table = [
28             ["Date", "Transaction", "Debit", "Credit", "Balance"],
29             ["01-02-2026", "UPI", "500.00", "", "9500.00"],
30             ["02-02-2026", "Salary", "", "2000.00", "11500.00"],
31         ]
32         page = MagicMock()
33         page.extract_tables.return_value = [table]
34         page.extract_text.return_value = "aubank statement"
35         page.extract_words.return_value = []
36         pdf = MagicMock()
37         pdf.pages = [page]
38         with patch("pdfplumber.open") as m:
39             m.return_value.__enter__.return_value = pdf

```

```

40         r = parse_au_savings(Path("x.pdf"), password="")
41         assert r.verified and len(r.rows) == 2
42         assert r.rows[0].debit == 500.0 and r.rows[1].credit == 2000.0
43
44
45     def test_parse_and_insert_stores_when_verified(tmp_path, monkeypatch):
46         rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0,
47 balance=9500.0)]
48         monkeypatch.setattr(au, "parse_au_savings", lambda *a, **k: SavingsParse("AU1",
49 rows, True))
50         with dbmod.connect(tmp_path / "t.db") as conn:
51             dbmod.init_db(conn)
52             doc_id = dbmod.insert_document(conn, source="file", sha256="au1",
53 status="parsed")
54             n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
55             assert n == 1
56             stored = dbmod.list_au_transactions(conn)
57             assert (
58                 len(stored) == 1
59                 and stored[0]["account_number"] == "AU1"
60                 and stored[0]["debit"] == 500.0
61             )
62             assert dbmod.get_document_by_sha(conn, "au1")["status"] == "reconciled"
63
64     def test_parse_and_insert_refuses_when_unverified(tmp_path, monkeypatch):
65         rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0,
66 balance=1.0)]
67         monkeypatch.setattr(au, "parse_au_savings", lambda *a, **k: SavingsParse("AU1",
68 rows, False))
69         with dbmod.connect(tmp_path / "t.db") as conn:
70             dbmod.init_db(conn)
71             doc_id = dbmod.insert_document(conn, source="file", sha256="au2",
72 status="parsed")
73             n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
74             assert n == 0
75             assert dbmod.list_au_transactions(conn) == []
76             assert dbmod.get_document_by_sha(conn, "au2")["status"] == "unreconciled"
77

```

32. user (2026-06-10T16:15:07.868Z)

```

1     """Local-only checks for the Axis savings parser against real encrypted statements.
2
3     Skipped unless an Axis password is available (`password_rules.yaml` or
4     `MUNEEM_TEST_AXIS_PW`); never runs in CI. The savings parser is **oracle-gated**
5     (coordinate clustering + `extract_tables` ? engine mapping ? reconciliation), so this
6     asserts the safety invariant: it either stored nothing (a vintage it can't yet read) or
7     everything it stored reconciled ? never mislabeled rows. Surfaces counts / booleans /
8     parser names only ? no statement content.
9     """
10
11     import os
12     from pathlib import Path
13
14     import pdfplumber
15     import pytest
16
17     from muneem import db as dbmod
18     from muneem.parsers import get_parser
19     from muneem.password_rules import get_password_for_bank
20
21     AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23

```

```

24     @pytest.mark.local_statements
25     @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26     def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27         pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28         if not pw:
29             pytest.skip("needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)")
30
31         with pdfplumber.open(pdf_path, password=pw) as pdf:
32             text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
33             parser = get_parser(text)
34             if parser is None:
35                 pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37             with dbmod.connect(tmp_path / "t.db") as conn:
38                 dbmod.init_db(conn)
39                 doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40                 n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41                 status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42                 stored = dbmod.list_axis_transactions(conn)
43
44                 # Oracle-gated invariant: the savings parser either stores nothing (and flags the
45                 # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46                 if type(parser).__name__ == "AxisParser":
47                     assert (n == 0 and stored == []) or status == "reconciled", (
48                         f"{pdf_path.name}: stored {n} row(s) with status={status} ? "
49                         "stop-gap must not persist unreconciled rows"
50                     )
51

```

33. user (2026-06-10T16:15:12.659Z)

```

1     """Every parser wraps an unexpected failure in a clean, non-leaking ``ParseError``.
2
3     A PDF library can raise with statement text in its message; the muneem-level error must
expose only
4     a redacted filename (no full path, no statement contents, no PII digits), while
preserving the
5     original cause via exception chaining for local debugging.
6     """
7
8     from pathlib import Path
9
10    import pytest
11
12    from muneem import db as dbmod
13    from muneem.errors import ParseError
14    from muneem.parsers import au, axis, hdfc
15
16    # Looks like statement text a pdf lib might surface in an exception message.
17    _SECRET = "ACCT 998877665544 balance 1,23,456.78"
18
19
20    def _raiser(*_a, **_k):
21        raise ValueError(_SECRET)
22
23
24    CASES = [
25        ("muneem.parsers.axis.parse_axis_savings", axis.AxisParser),
26        ("muneem.parsers.axis.parse_axis_cc", axis.AxisCreditCardParser),
27        ("muneem.parsers.au.parse_au_savings", au.AUParser),
28        ("muneem.parsers.hdfc.parse_statement", hdfc.HDFCParser),

```

```

29     ]
30
31
32     @pytest.mark.parametrize("patch_target, parser_cls", CASES)
33     def test_parse_failures_become_clean_parseerror(monkeypatch, patch_target, parser_cls):
34         monkeypatch.setattr(patch_target, _raiser)
35         parser = parser_cls()
36         pdf = Path("/home/avi/statements/Avi_Dullu_28022026.pdf")
37         with dbmod.connect(":memory:") as conn:
38             dbmod.init_db(conn)
39             with pytest.raises(ParseError) as ei:
40                 parser.parse_and_insert(pdf, conn, 1, password="pw")
41             msg = str(ei.value)
42             assert _SECRET not in msg and "998877665544" not in msg # no statement text
43             assert "/home/avi" not in msg and "statements" not in msg # no full path
44             assert "28022026" not in msg # PII digits masked
45             assert isinstance(ei.value.__cause__, ValueError) # original cause preserved
46

```

34. user (2026-06-10T16:15:13.122Z)

```

1     """Regression test for the parser registry (the PR #9-11 production-path bug).
2
3     `from muneem.parsers import get_parser` must return a working selector ? i.e. the
4     registry must be populated by package import alone, without anything importing the
5     axis/hdfc/... submodules first. We check in a FRESH interpreter (subprocess) so a
6     prior test importing a submodule can't mask a regression.
7     """
8
9     from __future__ import annotations
10
11     import subprocess
12     import sys
13
14
15     def test_registry_is_populated_by_package_import_alone():
16         code = (
17             "from muneem.parsers import get_parser, _REGISTRY\n"
18             "assert len(_REGISTRY) >= 2, f'registry empty: {_REGISTRY}'\n"
19             "assert get_parser('Axis Bank Statement of Account') is not None\n"
20             "assert get_parser('HDFC Bank savings account details') is not None\n"
21             "assert get_parser('a wholly unrelated document') is None\n"
22             "print('REGISTRY_OK', len(_REGISTRY))\n"
23         )
24         result = subprocess.run([sys.executable, "-c", code], capture_output=True,
text=True)
25         assert result.returncode == 0, f"fresh-interpreter import failed:\n{result.stderr}"
26         assert "REGISTRY_OK" in result.stdout
27
28
29     def test_get_parser_selects_by_sender():
30         from muneem.parsers import get_parser
31
32         assert type(get_parser("", sender="alerts@axisbank.com")).__name__ == "AxisParser"
33

```

35. user (2026-06-10T16:15:14.020Z)

```

1     """Local-only checks for the AU Bank parser against real encrypted statements.
2
3     Skipped unless an AU password is available (`password_rules.yaml` or
4     ``MUNEEM_TEST_AU_PW``); never
5     runs in CI. The parser is **oracle-gated**, so this asserts the safety invariant: it
6     either stored
7     nothing (a vintage it can't yet read) or everything it stored reconciled ? never

```



```

mislabeled rows.
6     Surfaces counts / booleans / parser names only ? no statement content.
7     ""
8
9     import os
10    from pathlib import Path
11
12    import pdfplumber
13    import pytest
14
15    from muneem import db as dbmod
16    from muneem.parsers import get_parser
17    from muneem.password_rules import get_password_for_bank
18
19    AU_PDFS = sorted(Path("testdata/au-bank-statements").glob("*.pdf"))
20
21
22    @pytest.mark.local_statements
23    @pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p: p.name)
24    def test_local_au_never_stores_unreconciled(pdf_path, tmp_path):
25        pw = (
26            get_password_for_bank("aubank")
27            or get_password_for_bank("au")
28            or os.environ.get("MUNEEM_TEST_AU_PW")
29        )
30        if not pw:
31            pytest.skip("needs the AU password (password_rules.yaml or MUNEEM_TEST_AU_PW)")
32
33        with pdfplumber.open(pdf_path, password=pw) as pdf:
34            text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
35len(pdf.pages))))
36            parser = get_parser(text)
37            if parser is None or type(parser).__name__ != "AUParser":
38                pytest.skip(f"{pdf_path.name}: not routed to AUParser")
39
40            with dbmod.connect(tmp_path / "t.db") as conn:
41                dbmod.init_db(conn)
42                doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
43status="parsed")
44                n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
45                status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
46                stored = dbmod.list_au_transactions(conn)
47
48                assert (n == 0 and stored == []) or status == "reconciled", (
49                    f"{pdf_path.name}: stored {n} row(s) with status={status} ? must not persist
unreconciled"
50                )
51

```

36. user (2026-06-10T16:15:18.381Z)

```

1     ""Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.
2
3     Generates *real* PDFs (no committed statements, no secrets) and runs the production
4     pipeline ?
5     ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
6     parse. This
7     covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
8     synthetic
9     word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP §
10    "Planned ?
11    Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud
12    CI.
13    ""
14

```

```

10     from pathlib import Path
11
12     from muneem import db as dbmod
13     from muneem.extract import extract
14     from muneem.parsers import get_parser
15     from synthpdf import at, build_pdf, right
16
17
18     def _hdfc_pdf(path: Path) -> Path:
19         """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount
columns whose
20         right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance
?490), plus a
21         SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing
10,000)."""
22         p = [
23             at(50, 60, "HDFC BANK"),
24             at(50, 72, "Savings Account Details"),
25             at(50, 84, "Account Number : 50100012345678"),
26             # header ? must carry "Narration" + a "Closing" token so _table_body locates the
table
27             at(72, 110, "Date"),
28             at(175, 110, "Narration"),
29             at(300, 110, "Withdrawals"),
30             at(390, 110, "Deposits"),
31             at(480, 110, "Closing"),
32             at(515, 110, "Balance"),
33         ]
34         rows = [
35             ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
36             ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
37             ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
38         ]
39         top = 130
40         for d, narr, wd, dep, bal in rows:
41             p += [
42                 at(52, top, d),
43                 at(111, top, narr),
44                 right(358, top, wd),
45                 right(443, top, dep),
46                 right(561, top, bal),
47             ]
48             top += 15
49         p += [
50             at(50, 200, "SUMMARY"),
51             at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
52             at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
53             at(50, 236, "Debit Count Credit Count"),
54             at(50, 248, "2 1"),
55         ]
56         return build_pdf(p, path)
57
58
59     def _au_savings_pdf(path: Path) -> Path:
60         """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
61         date | narration | debit | credit | balance table in well-separated columns ?
coordinate
62         clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening
50,000)."""
63         p = [
64             at(50, 60, "AU Small Finance Bank"),
65             at(50, 72, "Account Number : 1234567890123"),
66             # header (clustering's date+money filter drops it; here only for realism / text
markers)
67             at(52, 100, "Date"),

```

```

68         at(110, 100, "Transaction"),
69         at(310, 100, "Withdrawal"),
70         at(395, 100, "Deposit"),
71         at(485, 100, "Balance"),
72     ]
73     rows = [
74         ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
75         ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
76         ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
77     ]
78     top = 124
79     for d, narr, debit, credit, bal in rows:
80         line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
81         if debit:
82             line.append(right(340, top, debit))
83         if credit:
84             line.append(right(420, top, credit))
85         p += line
86         top += 15
87     return build_pdf(p, path)
88
89
90     def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
91         """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back
92         status."""
93         ex = extract(pdf)
94         parser = get_parser(ex.text)
95         assert parser is not None, "no parser routed this synthetic statement"
96         with dbmod.connect(":memory:") as conn:
97             dbmod.init_db(conn)
98             doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256,
99             status="parsed")
100             n = parser.parse_and_insert(pdf, conn, doc_id)
101             status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
102         return type(parser).__name__, n, status
103
104     def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
105         name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
106         assert name == "HDFCParser"
107         assert n == 3, f"expected 3 transactions, got {n}"
108         assert status == "reconciled", f"expected reconciled, got {status}"
109
110     def test_savings_layout_full_pipeline_reconciles(tmp_path):
111         name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
112         assert name == "AUParser"
113         assert n == 3, f"expected 3 transactions, got {n}"
114         assert status == "reconciled", f"expected reconciled, got {status}"
115

```

37. user (2026-06-10T16:15:25.205Z)

```

1     """Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.
2
3     DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:
4
5     1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier
6     successful
7     unlocks) plus those configured in ``password_rules.yaml`` (see
8     :mod:`muneem.password_rules`).
9     2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with
10    ``interactive=False``,
11    e.g. automation / CI).

```

```

11     A password that works **via the prompt** is cached in the keychain so the next ingest is
12     friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the
cache is
13     the OS keychain (:mod:`muneem.secrets`), never a file.
14     """
15
16     from __future__ import annotations
17
18     import getpass
19     import json
20     from pathlib import Path
21
22     from muneem import secrets
23     from muneem.errors import KeychainError
24     from muneem.password_rules import candidate_passwords
25     from muneem.unlock import is_encrypted, unlock
26
27     # Keychain entry holding a JSON list of known-good statement passwords (the learned
candidates).
28     _CACHE_KEY = "statement_unlock_candidates"
29     # Bound the cached list so the keychain entry can't grow without limit; most-recently-
used kept.
30     _MAX_CACHED = 20
31
32
33     def _cached_candidates() -> list[str]:
34         raw = secrets.get_secret(_CACHE_KEY)
35         if not raw:
36             return []
37         try:
38             return [p for p in json.loads(raw) if isinstance(p, str) and p]
39         except (ValueError, TypeError):
40             return []
41
42
43     def _remember(password: str) -> None:
44         """Cache a working password in the keychain (MRU order, deduped, capped). Never
logged.
45
46         Best-effort: a keychain write failure is swallowed (the password already unlocked
the file this
47         run); we never crash an ingest over a caching hiccup.
48         """
49         if not password or not password.strip():
50             return
51         current = [p for p in _cached_candidates() if p != password]
52         updated = [password, *current][:_MAX_CACHED] # most-recently-used first, bounded
53         try:
54             secrets.set_secret(_CACHE_KEY, json.dumps(updated))
55         except KeychainError:
56             pass
57
58
59     def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
60         """Return a working password for ``pdf_path``.
61
62         ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
encrypted and
63         no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
time.
64         """
65         path = Path(pdf_path)
66         if not is_encrypted(path):
67             return ""
68

```

```

69     candidates = [*_cached_candidates(), *candidate_passwords()]
70     prompt_fn = None
71     if interactive:
72
73         def prompt_fn() -> str | None: # noqa: E306 ? local prompt; no echo, never
logged
74             return getpass.getpass(f"Password for {path.name}: ") or None
75
76     result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
77     if not result.ok:
78         return None
79     if result.method == "prompt" and result.password:
80         _remember(result.password)
81     return result.password or ""
82

```

38. user (2026-06-10T16:15:25.632Z)

```

1     """Secret storage via the OS keychain.
2
3     OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4     Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5     in logs. See DESIGN.md section 5.
6
7     External-call discipline (every keyring call is wrapped with attribution):
8     - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
`None`
9     (absence is a valid state; this also keeps headless CI happy).
10    - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11    because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12    included for diagnosis; the value never is.
13    """
14
15    from __future__ import annotations
16
17    import keyring
18    from keyring.errors import KeyringError, PasswordDeleteError
19
20    from muneem.errors import KeychainError
21
22    _SERVICE = "muneem"
23
24
25    def get_secret(name: str) -> str | None:
26        """Return the stored secret, or `None` if absent or the keychain is
unavailable."""
27        try:
28            return keyring.get_password(_SERVICE, name)
29        except KeyringError:
30            return None
31
32
33    def set_secret(name: str, value: str) -> None:
34        """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35        try:
36            keyring.set_password(_SERVICE, name, value)
37        except KeyringError as exc:
38            # Name only ? never the value ? in the message (it may be a password or the DB
key).
39            raise KeychainError(f"could not store secret {name!r} in the OS keychain") from

```

```

exc
40
41
42     def delete_secret(name: str) -> None:
43         """Delete a secret if present; a no-op if it is not (or the keychain is
44         unavailable)."""
45         try:
46             keyring.delete_password(_SERVICE, name)
47         except (PasswordDeleteError, KeyringError):
48             pass

```

39. user (2026-06-10T16:15:33.286Z)

docs\CURRENT_STATE.md:30:[Omitted long matching line]

docs\CURRENT_STATE.md:55:* **Recover the May 2025 vintage** : the clusterer drops one transaction line ? incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing; June 2025 was recovered via balance-delta reconstruction).

docs\parser-architecture.md:118:sign errors, dropped/duplicated rows, and column misalignment** ? which almost always **break the

docs\ROADMAP.md:36:> **Remaining:** recover the dropped line (smarter line recovery in the clusterer) for May 2025 and

docs\ROADMAP.md:48:2. **Axis May 2025 recovery** ? recover the one transaction line the clusterer drops (incomplete

40. user (2026-06-10T16:15:33.616Z)

```

1     """Unit tests for the Gmail provider using a hand-rolled fake service."""
2
3     import base64
4
5     from muneem.ingestion.base import SearchCriteria
6     from muneem.ingestion.gmail import GmailProvider
7
8
9     def _b64(text: str) -> str:
10         return base64.urlsafe_b64encode(text.encode()).decode()
11
12
13     class _Resp:
14         def __init__(self, data):
15             self._data = data
16
17         def execute(self):
18             return self._data
19
20
21     class _Attachments:
22         def __init__(self, data):
23             self._data = data
24
25         def get(self, userId, messageId, id):
26             return _Resp({"data": self._data["attachments"][id]})
27
28
29     class _Messages:
30         def __init__(self, data):
31             self._data = data
32
33         def list(self, userId, q, pageToken=None, maxResults=100):
34             return _Resp(self._data["list"])
35
36         def get(self, userId, id, format=None):
37             return _Resp(self._data["messages"][id])
38

```

```

39         def attachments(self):
40             return _Attachments(self._data)
41
42
43     class _Users:
44         def __init__(self, data):
45             self._data = data
46
47         def messages(self):
48             return _Messages(self._data)
49
50
51     class FakeService:
52         def __init__(self, data):
53             self._data = data
54
55         def users(self):
56             return _Users(self._data)
57
58
59     def _make_data():
60         pdf_bytes = b"%PDF-1.4 fake"
61         body_data = _b64("Your password is your Customer ID.")
62         message = {
63             "payload": {
64                 "headers": [
65                     {"name": "Subject", "value": "HDFC Bank Statement"},
66                     {"name": "Date", "value": "Mon, 28 Feb 2026 18:38:54 +0530"},
67                 ],
68                 "parts": [
69                     {"mimeType": "text/plain", "body": {"data": body_data}},
70                     {"filename": "statement.pdf", "body": {"attachmentId": "att1"}},
71                 ],
72             }
73         }
74         return {
75             "list": {"messages": [{"id": "m1"}, {"id": "m2"}]},
76             "messages": {"m1": message, "m2": message},
77             "attachments": {"att1": base64.urlsafe_b64encode(pdf_bytes).decode()},
78         }
79
80
81     def test_search_message_ids():
82         provider = GmailProvider("dummy")
83         provider.service = FakeService(_make_data())
84         criteria = SearchCriteria(sender="anything", has_attachment=True,
newer_than_days=None)
85         assert provider.search_messages(criteria) == ["m1", "m2"]
86
87
88     def test_get_headers():
89         provider = GmailProvider("dummy")
90         provider.service = FakeService(_make_data())
91         headers = provider.get_headers("m1")
92         assert headers["subject"] == "HDFC Bank Statement"
93
94
95     def test_list_pdf_attachments():
96         provider = GmailProvider("dummy")
97         provider.service = FakeService(_make_data())
98         assert provider.list_pdf_attachments("m1") == [("statement.pdf", "att1")]
99
100
101     def test_download_attachment(tmp_path):
102         provider = GmailProvider("dummy")

```

```

103     provider.service = FakeService(_make_data())
104     out = provider.download_attachment("m1", "att1", tmp_path / "statement.pdf")
105     assert out.read_bytes() == b"%PDF-1.4 fake"
106

```

41. user (2026-06-10T16:15:40.327Z)

```

1      # Current State
2
3      > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4      > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5      > See [ROADMAP.md](./ROADMAP.md) for milestone tracking and [parser-
architecture.md](./parser-architecture.md) for the parser design.
6
7      > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8      > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9      > **targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10     > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
instrument
11     > family is *new data + a reconciliation rule*, not a new pipeline.
12
13     ## 1. Core Ingestion Pipeline
14     The ingestion pipeline supports a generic SearchCriteria interface and multiple
providers (Track B plumbing):
15
16     * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17     * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18     * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20     Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22     ## 2. Statement Parsers (concept-core architecture)
23     Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`_autoload_parsers`), so the registry is populated in the production path (not just under
tests).
24
25     The parser core is **concept-first and verification-first** (see [`parser-
architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-
agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the
column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a
shared **verifier** (`validation.py`) decides trust by *arithmetic* ? per-row balance walk +
total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-
agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future
**instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the
same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as
**data, not code**. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-
table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly
a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).
26
27     **Active issuers** ? we have real statements + passwords, so these get validated on real
data:
28
29     * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate
clustering** (deliberately *not* `extract_tables` ? HDFC's table is semi-ruled). **Self-
validating:** running-balance reconciliation + a cross-check against the bank's own SUMMARY

```


(Dr/Cr counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 6 real statements** ? all reconcile **and** bookend, without inspecting any value.

30 * **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == [?balance]`. Oracle-gated, so it never persists unreconciled rows. ? **6/7 real statements reconcile + persist**; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).

31 * **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just **AUBANK_PROFILE**. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

32

33 > **_Dropped: ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

34

35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 173 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution is now wired into `muneem ingest --doc-type bank\_statement`** ? the hybrid unlock (keychain-cached + `password\_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement\_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

36

37 See `password\_rules.py` (prefers user config dir) and `cli.py:ingest`.

38

39 **## 3. Database & CLI**

40 * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis\_`, `hdfc\_transactions`) ? documented in [schema.md](./schema.md). Dedup on sha256. **Versioned migrations** via a hand-rolled runner (`muneem/migrations.py`): the v4 schema is the frozen baseline, forward migrations layer on (recorded in `schema\_migrations` with checksums), drift is rejected ? see [schema.md](./schema.md) and feedbackJune2026 P6. A unified `transactions` table (plus `holdings` for instruments) with per-row provenance/confidence is the planned direction once the design decision lands (parser-architecture.md §9; feedbackJune2026 P3).

41 * **The on-disk ledger is encrypted at rest** (decision D.2) ? **SQLCipher** transparent AES-256 via `sqlcipher3`, opened through `db.connect\_encrypted()`; the 256-bit key lives in the OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext stdlib `sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).

42 * CLI: `ingest <pdf>` (promo, or `--doc-type bank\_statement` ? **auto-unlocks encrypted PDFs**, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list <bank>`, `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`, `rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-at-rest.md)).

43 * Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement passwords, **the DB encryption key**) via keyring.

44

45 **## 4. Testing & Hygiene**

46 * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine fixtures (default CI).

47 * Security guards in `test\_repo\_hygiene.py` + license allowlist test.

48 * **DB invariant tripwires** (`test\_db\_invariants.py`, default CI): a **schema lock** (pins `SCHEMA\_VERSION` + every table/column shape ? an accidental drift fails the merge; a deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption guards** (production ingest must write an **encrypted** ledger; a keyed open never silently downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).

49 * **Local git hooks** (`.github/hooks/`, enable via `make hooks`): **pre-commit** blocks staging sensitive files; **pre-push** runs ruff + the offline suite (incl. the invariant

```

tripwires) so only green code reaches GitHub. The pre-push gate is the local stand-in for
server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays
private). Both are bypassable with --no-verify (the conscious "explicitly allowed" escape).
50      * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed). The suite never touches the real OS keychain (autouse fixture in
`tests/conftest.py`) ? headless-CI safe.
51
52      ## 5. Next Steps
53      Concept-core steps 1?3 are done**, and password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: HDFC 6/6, AU 4/4, Axis
6/7 (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55      * Recover the May 2025 vintage: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
56      * Step 4 ? L3 local-vision fallback (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
57      * Step 5 ? unify storage (`transactions` + `holdings`) and begin the instrument
family (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
58      * Regression CI for statements (decided 2026-06, deferred ? not yet built): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) ($ "Planned ? Regression CI for statement parsing").
59
60      Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher;
see [encryption-at-rest.md](./encryption-at-rest.md))._
61
62      The project stays local-first, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
63

```

42. user (2026-06-10T16:15:50.424Z)

```

testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
---
--- log check ---

```

43. user (2026-06-10T16:15:50.437Z)

```

1      """Decrypt password-protected statement PDFs (Stage 2).
2
3      Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
4      password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
5      fall back to an interactive prompt. The working password is returned so the
6      caller can cache it in the OS keychain (muneem.secrets); it is never logged.
7      """
8
9      from __future__ import annotations
10
11     import shutil
12     from collections.abc import Callable, Iterable

```

```

13 from dataclasses import dataclass
14 from pathlib import Path
15
16 import pypdf
17
18
19 @dataclass
20 class UnlockResult:
21     ok: bool
22     method: str | None # "plaintext" | "candidate" | "prompt"
23     password: str | None # working password ? do NOT log; None if plaintext/failed
24     output_path: Path | None # decrypted copy, if a destination was given
25
26
27 def is_encrypted(path: Path | str) -> bool:
28     return pypdf.PdfReader(str(path)).is_encrypted
29
30
31 def password_works(path: Path | str, password: str) -> bool:
32     reader = pypdf.PdfReader(str(path))
33     if not reader.is_encrypted:
34         return True
35     try:
36         return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
37     except Exception:
38         return False
39
40
41 def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
42     """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
43     src = Path(path)
44     dest = Path(dest)
45     dest.parent.mkdir(parents=True, exist_ok=True)
46     reader = pypdf.PdfReader(str(src))
47     if reader.is_encrypted:
48         reader.decrypt(password or "")
49     writer = pypdf.PdfWriter()
50     writer.append(reader)
51     with open(dest, "wb") as fh:
52         writer.write(fh)
53     return dest
54
55
56 def unlock(
57     path: Path | str,
58     candidates: Iterable[str] = (),
59     prompt_fn: Callable[[], str | None] | None = None,
60     max_prompts: int = 3,
61     dest: Path | str | None = None,
62 ) -> UnlockResult:
63     """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.
64
65     If dest is given and unlock succeeds, a decrypted copy is written there. The
66     working password is returned in the result but must never be logged.
67     """
68     src = Path(path)
69     if not is_encrypted(src):
70         out = Path(shutil.copyfile(src, dest)) if dest else None
71         return UnlockResult(True, "plaintext", None, out)
72
73     tried: set[str] = set()
74     for pw in candidates:
75         if pw in tried:
76             continue
77         tried.add(pw)

```

```

78         if password_works(src, pw):
79             out = write_decrypted(src, pw, dest) if dest else None
80             return UnlockResult(True, "candidate", pw, out)
81
82     if prompt_fn is not None:
83         for _ in range(max_prompts):
84             entered = prompt_fn()
85             if not entered:
86                 break
87             if password_works(src, entered):
88                 out = write_decrypted(src, entered, dest) if dest else None
89                 return UnlockResult(True, "prompt", entered, out)
90
91     return UnlockResult(False, None, None, None)
92

```

44. user (2026-06-10T16:20:17.157Z)

DESIGN.md:46:### Stage 1 ? Email ingestion

DESIGN.md:162:| **Go only** | Single binary, easy distribution, great concurrency for the ingestion side, the user already uses Go for the badminton-highlight-indexer project | Weak PDF parsing / OCR / ML ecosystem; rolling our own is painful |

DESIGN.md:163:| **Python only** | Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF, Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype | Heavier deployment story; less ergonomic for the long-running ingestion daemon |

DESIGN.md:164:| **Go ingestion + Python parsing** | Plays to each language's strengths | Two-language project complexity; inter-process boundary to maintain |

DESIGN.md:166:Recommended default: **Python-only** for v1, accept the deployment-story cost in exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.

DESIGN.md:186:> **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus parse?store?query spine **and** the Gmail ingestion plumbing, as two decoupled tracks that converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a single end-to-end bank slice first). See [docs/ROADMAP.md](./docs/ROADMAP.md) §§ 1 and 4.

pyproject.toml:76:# The email-ingestion providers (gmail/imap/outlook) are experimental scaffolding with

pyproject.toml:80:module = "muneem.ingestion."

tools\gmail_auth.py:45: from muneem.ingestion.base import SearchCriteria

tools\gmail_auth.py:46: from muneem.ingestion.gmail import SCOPES, GmailProvider

tools\gmail_auth.py:51: provider = GmailProvider(creds_path)

tools\imap_auth.py:22: from muneem.ingestion.base import SearchCriteria

tools\imap_auth.py:23: from muneem.ingestion.imap import IMAPProvider

tools\imap_auth.py:28: provider = IMAPProvider(args.host, args.user)

tools\outlook_auth.py:23: from muneem.ingestion.base import SearchCriteria

tools\outlook_auth.py:24: from muneem.ingestion.outlook import SCOPES, OutlookProvider

tools\outlook_auth.py:29: provider = OutlookProvider(args.client_id)

docs\CURRENT_STATE.md:14:The ingestion pipeline supports a generic `SearchCriteria` interface and multiple providers (Track B plumbing):

docs\CURRENT_STATE.md:16: * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment download implemented.

docs\CURRENT_STATE.md:17: * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get\_headers` / attachment methods are NotImplemented (experimental).

docs\CURRENT_STATE.md:18: * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented; attachment/header/download NotImplemented (experimental).

docs\DEPENDENCIES.md:77:**Optional deps** (outlook extra): msal + requests ? only loaded when OutlookProvider is

docs\feedbackJune2026.md:82:`ingestion` at sensible levels; keep `RedactingFilter` as the **backstop**, not the primary control

docs\feedbackJune2026.md:194:- **Phase F** ? `muneem fetch` + ingestion completeness (P7).

docs\ROADMAP.md:66:| § 8.1 Language | Implementation language for v1 | **Python-only**. Accept the heavier deployment story in exchange for the best PDF/OCR/LLM ecosystem and fastest path on the hard parsing work. Revisit only if the ingestion daemon outgrows Python. |

docs\ROADMAP.md:67:| § 8.4 First slice | What to build first | **Both tracks in parallel** ? the promo-corpus parse?store?query spine **and** the Gmail ingestion plumbing, developed as two decoupled tracks that converge at the first bank slice. |

docs\ROADMAP.md:114: TRACK A: Promo spine TRACK B: Gmail ingestion (run in

parallel,
docs\ROADMAP.md:157:### Track B ? Gmail ingestion · PARTIAL (OAuth + search + download ?;
`muneem fetch` B.4 pending)

45. user (2026-06-10T16:20:17.461Z)

```
1      """Hand-rolled, versioned, checksummed schema migrations (feedbackJune2026 P6).
2
3      Deliberately **not Alembic** ? muneem stays raw-`sqlite3` / stdlib-light. Migrations are
an ordered
4      list; each is applied exactly once, *inside* the (possibly encrypted) connection, and
recorded in a
5      ``schema_migrations`` table with a checksum. The schema-lock test
(`tests/test_db_invariants.py`)
6      pins the resulting shape, so accidental drift and deliberate changes both surface in
review.
7
8      Model:
9      - **Baseline.** The current schema (`db._SCHEMA`, version `db.SCHEMA_VERSION`) is the
*frozen*
10     baseline ? never edit it for new changes; add a migration instead. A fresh DB gets the
baseline
11     applied; a legacy DB created before this framework (tables present, no
`schema_migrations`) is
12     detected and its baseline row back-filled (no DDL re-run).
13     - **Forward migrations.** Entries in `MIGRATIONS` with ``version > baseline`` run in
order. (None
14     yet ? the first lands with the unified-schema work, feedbackJune2026 P3.)
15
16     Known limitation (single-user, reviewed migrations): a crash *between* applying a
migration's DDL
17     and recording it could require manual cleanup; per-migration transactions can be added
when the
18     first non-trivial migration lands.
19     """
20
21     from __future__ import annotations
22
23     import hashlib
24     import sqlite3
25     from dataclasses import dataclass
26
27     from muneem.errors import MigrationError
28
29     _SCHEMA_MIGRATIONS = """
30     CREATE TABLE IF NOT EXISTS schema_migrations (
31         version    INTEGER PRIMARY KEY,
32         name       TEXT NOT NULL,
33         checksum   TEXT NOT NULL,
34         applied_at TEXT NOT NULL DEFAULT (datetime('now'))
35     );
36     """
37
38
39     @dataclass(frozen=True)
40     class Migration:
41         """One forward migration: a version, a name, and the DDL to apply (run via
executescript)."""
42
43         version: int
44         name: str
45         sql: str
46
47         @property
48         def checksum(self) -> str:
```

```

49         return hashlib.sha256(self.sql.encode("utf-8")).hexdigest()
50
51
52     # --- Forward migrations (version > baseline), applied in ascending order
53     -----
54     # v5: provenance/confidence as a FIRST-CLASS but SEPARATE table (DESIGN §8.7,
55     # feedbackJune2026 P3).
56     # Keyed by document_id (1:1) so every txn row reaches its provenance by the existing FK
57     # join ? no
58     # per-row redundancy. source_type readies it for non-PDF sources (aa/sms) we may add
59     # later.
60     _M5_PROVENANCE = """
61     CREATE TABLE IF NOT EXISTS provenance (
62         id                INTEGER PRIMARY KEY,
63         document_id        INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,
64         source_type        TEXT NOT NULL,
65         parser             TEXT,
66         parser_version     TEXT,
67         mapping_layer     TEXT,
68         confidence        TEXT,
69         created_at        TEXT NOT NULL DEFAULT (datetime('now'))
70     );
71     """
72
73     MIGRATIONS: list[Migration] = [
74         Migration(version=5, name="provenance", sql=_M5_PROVENANCE),
75     ]
76
77     def _checksum(sql: str) -> str:
78         return hashlib.sha256(sql.encode("utf-8")).hexdigest()
79
80     def _user_tables(conn: sqlite3.Connection) -> list[str]:
81         return [
82             r[0]
83             for r in conn.execute(
84                 "SELECT name FROM sqlite_master WHERE type='table' "
85                 "AND name NOT LIKE 'sqlite_%' AND name != 'schema_migrations'"
86             )
87         ]
88
89     def _record(conn: sqlite3.Connection, version: int, name: str, checksum: str) -> None:
90         conn.execute(
91             "INSERT INTO schema_migrations (version, name, checksum) VALUES (?, ?, ?)",
92             (version, name, checksum),
93         )
94
95     def run_migrations(
96         conn: sqlite3.Connection,
97         *,
98         baseline_sql: str,
99         baseline_version: int,
100         migrations: list[Migration] | None = None,
101     ) -> int:
102         """Bring ``conn``'s schema up to date. Returns the resulting ``user_version``.
103
104         Idempotent: re-running applies nothing already recorded. Raises
105         :class:`MigrationError` if the
106         frozen baseline's checksum no longer matches what was recorded (i.e. someone edited
107         the baseline
108         instead of adding a migration).

```

```

108     """
109     migrations = MIGRATIONS if migrations is None else migrations
110     conn.executescript(_SCHEMA_MIGRATIONS) # bookkeeping table first (idempotent)
111     applied = {r[0] for r in conn.execute("SELECT version FROM schema_migrations")}
112     current = conn.execute("PRAGMA user_version").fetchone()[0]
113     baseline_checksum = _checksum(baseline_sql)
114
115     if baseline_version in applied:
116         recorded = conn.execute(
117             "SELECT checksum FROM schema_migrations WHERE version = ?",
118             (baseline_version,)
119         ).fetchone()
120         if recorded and recorded[0] != baseline_checksum:
121             raise MigrationError(
122                 "baseline schema changed without a migration ? add a migration instead
of editing "
123                 "the frozen baseline (feedbackJune2026 P6)"
124             )
125         else:
126             # Fresh DB ? create the baseline; legacy DB (tables already present) ? just
record it.
127             if not _user_tables(conn):
128                 conn.executescript(baseline_sql)
129             _record(conn, baseline_version, "baseline", baseline_checksum)
130             if current < baseline_version:
131                 conn.execute(f"PRAGMA user_version = {baseline_version}")
132                 current = baseline_version
133
134             for migration in sorted(migrations, key=lambda m: m.version):
135                 if migration.version <= baseline_version or migration.version in applied:
136                     continue
137                 conn.executescript(migration.sql)
138                 _record(conn, migration.version, migration.name, migration.checksum)
139                 conn.execute(f"PRAGMA user_version = {migration.version}")
140                 current = migration.version
141
142     return int(current)

```

46. user (2026-06-10T16:20:35.967Z)

```

1     """Password rule loader for statement decryption (hybrid strategy support).
2
3     Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4     on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
5     dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
6     .gitignore`
7     blocks `password_rules.*`.
8
9     See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
file
10    format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [...]``
list.
11    """
12    from __future__ import annotations
13
14    import warnings
15    from pathlib import Path
16    from typing import Any
17
18    import yaml
19
20    from muneem.config import CONFIG_DIR
21    from muneem.errors import ConfigError

```

```

22
23
24 def _load_rules() -> dict[str, Any]:
25     """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
absent.
26
27     Resolution order:
28     1. ``<config_dir>/password_rules.yaml`` (recommended location)
29     2. ``./password_rules.yaml`` (legacy, emits DeprecationWarning)
30     """
31     for path, is_legacy in (
32         (CONFIG_DIR / "password_rules.yaml", False),
33         (Path("password_rules.yaml"), True),
34     ):
35         if path.exists():
36             if is_legacy:
37                 warnings.warn(
38                     "Loading password_rules from CWD (./password_rules.yaml) is
deprecated. "
39                     f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
proofing. "
40                     "See docs and CLAUDE.md (never commit real password material).",
41                     DeprecationWarning,
42                     stacklevel=3,
43                 )
44             with path.open(encoding="utf-8") as f:
45                 try:
46                     data = yaml.safe_load(f)
47                 except yaml.YAMLError as exc:
48                     # Never echo the file's contents in the error ? they are password
material.
49                     raise ConfigError(
50                         f"could not parse password rules ({path.name}): invalid YAML"
51                     ) from exc
52                 if data is None:
53                     return {}
54                 if not isinstance(data, dict):
55                     raise ConfigError(
56                         f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
57                     )
58                 return data
59     return {}
60
61
62 def get_password_for_bank(bank_name: str) -> str | None:
63     """Return the password (or derived rule string) for a bank, or ``None``."""
64     val = _load_rules().get(bank_name)
65     return str(val) if val else None
66
67
68 def candidate_passwords() -> list[str]:
69     """Every configured password to try as an unlock candidate, **bank-agnostic**.
70
71     The ingest path resolves the password *before* it knows the issuer (it needs the
decrypted
72     text to pick a parser), so it tries all configured passwords. Collects each per-bank
value
73     plus any entries under a top-level ``candidates`` list; order preserved, de-
duplicated.
74     Quote numeric passwords in the YAML so leading zeros survive.
75     """
76     rules = _load_rules()
77     collected: list[str] = []
78     for key, val in rules.items():

```



```

79         if key == "candidates" and isinstance(val, list):
80             collected.extend(str(v) for v in val if v not in (None, ""))
81         elif isinstance(val, str | int | float):
82             collected.append(str(val))
83     seen: set[str] = set()
84     unique: list[str] = []
85     for pw in collected:
86         if pw and pw not in seen:
87             seen.add(pw)
88             unique.append(pw)
89     return unique
90

```

47. user (2026-06-10T16:20:36.264Z)

```

1     """Authorize Gmail (read-only) and list the statement emails found.
2
3     Run by *you* ? it opens a browser for Google consent, caches the token in your OS
4     keychain, then prints the statement emails it can see. Nothing is uploaded; this
5     only reads your inbox. Use it to confirm your OAuth client + consent + search work
6     before the full import pipeline.
7
8     python tools/gmail_auth.py --credentials testdata/credentials/client_secret_*.json
9     """
10
11     from __future__ import annotations
12
13     import argparse
14     import glob
15     import sys
16     from pathlib import Path
17
18
19     def _find_credentials(arg: str | None) -> Path | None:
20         if arg:
21             return Path(arg)
22         matches = sorted(glob.glob("testdata/credentials/*.json"))
23         return Path(matches[0]) if matches else None
24
25
26     def main() -> int:
27         parser = argparse.ArgumentParser(
28             description="Authorize Gmail (read-only) and list statement emails."
29         )
30         parser.add_argument("--credentials", help="Path to the OAuth client secret JSON.")
31         parser.add_argument("--sender", default="hdfcbank.net", help="Sender email domain to
32 search.")
33         parser.add_argument("--max", type=int, default=25, help="Max emails to list.")
34         args = parser.parse_args()
35
36         creds_path = _find_credentials(args.credentials)
37         if not creds_path or not creds_path.exists():
38             print(
39                 "No OAuth client secret found. Pass --credentials <path>. "
40                 "(Create a Desktop OAuth client in Google Cloud Console and download the
41 JSON; "
42                 "see README § Getting started.)",
43                 file=sys.stderr,
44             )
45             return 2
46
47         from muneem.ingestion.base import SearchCriteria
48         from muneem.ingestion.gmail import SCOPES, GmailProvider
49
50         print(f"Client secret : {creds_path.name}")

```

```

49         print(f"Scope          : {SCOPES[0]} (read-only ? cannot modify or delete mail)")
50
51         provider = GmailProvider(creds_path)
52         provider.authorize()
53
54         criteria = SearchCriteria(sender=args.sender, has_attachment=True,
newer_than_days=None)
55         print(f"Searching      : from:{args.sender} has:attachment")
56
57         ids = provider.search_messages(criteria, max_results=args.max)
58         print(f"Found {len(ids)} message(s) with a PDF attachment:\n")
59         for mid in ids:
60             hdrs = provider.get_headers(mid)
61             attachments = ", ".join(name for name, _ in provider.list_pdf_attachments(mid))
or "(none)"
62             date = hdrs.get("date", "")[:16]
63             subject = hdrs.get("subject", "(no subject)")[:55]
64             print(f" {date:16} {subject:55} {attachments}")
65         print("\nToken cached in your OS keychain. Done.")
66         return 0
67
68
69     if __name__ == "__main__":
70         raise SystemExit(main())
71

```

48. assistant (2026-06-10T16:20:50.679Z)

Now let me verify a few suspected failure modes empirically with the installed package.

49. user (2026-06-10T16:20:53.312Z)

.claude	.git	.githubhooks	.github	.mypy_cache
.pytest_cache				
.ruff_cache	.venv	docs	src	testdata
tests				
tools	.coverage	.gitattributes	.gitignore	CLAUDE.md
DESIGN.md				
Makefile	pyproject.toml	README.md		

True

50. user (2026-06-10T16:21:03.815Z)

registry order: ['AUParser', 'AxisParser', 'AxisCreditCardParser', 'HDFCParser']
routed to: AUParser

51. user (2026-06-10T16:21:17.930Z)

incomplete ledger endorsed: True
first-row misread endorsed: True

52. user (2026-06-10T16:21:33.830Z)

verified: False layer: L0 map: {'txn_date': 0, 'narration': 1, 'debit': 2, 'credit': 3, 'balance': 4}
stored row: 01-02-2026 debit= 500.0 credit= None balance= 1500.0
stored row: 02-02-2026 debit= 300.0 credit= None balance= 1800.0

53. user (2026-06-10T16:21:56.506Z)

verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 3, 'credit': 2, 'balance': 4}
stored: 01-02-2026 UPI OUT debit= None credit= 500.0 bal= 1500.0

stored: 02-02-2026 REFUND debit= 300.0 credit= None bal= 1200.0
stored: 03-02-2026 EMI OUT debit= None credit= 800.0 bal= 2000.0

54. assistant (2026-06-10T16:22:13.667Z)

Two failure modes confirmed. Let me check the one-paisa tolerance boundary behavior empirically.

55. user (2026-06-10T16:22:21.737Z)

one-paisa corruptions UNDETECTED: 8400/20000
5000-row float total still reconciles: True

56. user (2026-06-10T16:24:22.006Z)

```
src\muneem\parsers\__init__.py:3:from .base import StatementParser, safe_amount
src\muneem\parsers\__init__.py:44:__all__ = ["StatementParser", "register_parser", "get_parser",
"safe_amount"]
src\muneem\parsers\base.py:8:def safe_amount(s: str) -> float | None:
src\muneem\parsers\base.py:38:__all__ = ["StatementParser", "safe_amount"]
src\muneem\parsers\hdfc.py:115:    Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not
None): a zero amount
src\muneem\parsers\validation.py:40:    *absent* from *zero* ? unlike ``base.safe_amount``,
which maps ``"0.00"`` to
```

57. user (2026-06-10T16:24:22.809Z)

```
src\muneem\parsers\engine.py:303:def map_table(
src\muneem\parsers\savings.py:196:    res = map_table(table, profile, oracle=oracle)
src\muneem\parsers\savings.py:203:    clustered = map_table(matrix, profile, oracle=oracle)
```

58. assistant (2026-06-10T16:25:39.604Z)

All verification complete. I have everything needed for the structured report.

59. user (2026-06-10T16:27:30.463Z)

Structured output provided successfully

60. assistant (2026-06-10T16:27:42.268Z)

Review complete. Findings delivered via structured output: 3 high-severity (savings oracle ignoring available bookends, Cr/Dr sign-stripping enabling a verified-but-inverted overdraft parse, HDFC persisting unreconciled rows), 6 medium (one-paisa tolerance gap, substring routing false positives, dedup blocking re-parse, unverified Axis CC path, absent date normalization, clustering line drops), and 4 low items ? with the key failure modes empirically confirmed against the installed package using synthetic data only.